Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра штучного інтелекту

Дисципліна «Аналітика великих даних»

Практичне заняття №4

«Зберігання та аналіз великих даних у NoSQL-сховищі MongoDB»

Виконав:
ст. гр. ДМм-26-1
Зелінський В.С.

Прийняв:
Золотухін О.В.

Харків – 2026

Зберігання та аналіз великих даних у NoSQL-сховищі MongoDB

Мета роботи

Ознайомитися з принципами організації NoSQL-сховищ та документною моделлю MongoDB; отримати практичні навички створення бази даних і колекцій, імпорту великого набору даних, виконання запитів, індексації та агрегування на прикладі 7 816 537 хвилинних свічок BTC/USD (Bitcoin Historical Data, 2012–2026 роки); навчитися використовувати MongoDB як сховище результатів обробки великих даних, отриманих у ПЗ2 (Spark) та ПЗ3 (виявлення аномалій).

Короткі теоретичні відомості

NoSQL-системи відмовляються від жорсткої реляційної схеми на користь моделей, що легше масштабуються горизонтально; основні типи: key-value (значення за ключем, наприклад Redis), документні (MongoDB), колонкові (Cassandra, HBase) та графові (Neo4j).

Ієрархія MongoDB така: сервер `mongod` містить бази даних, база – колекції, колекція – документи. Документ зберігається у форматі BSON (двійковий JSON), який має типи, відсутні в JSON: Date (мілісекунди від епохи, UTC), Int32 і Int64, Double, ObjectId, масиви та вкладені документи. Розмір документа обмежено 16 МіБ, кожен документ має унікальне поле `_id`. Схема не нав'язується: документи однієї колекції можуть мати різні поля.

Запит `find(filter, projection)` використовує оператори порівняння (`$gt`, `$lt`, `$in`) і логічні (`$and`, `$or`). Aggregation Framework будує конвеєр стадій (`$match`, `$group`, `$sort`, `$project`, `$bucket`, `$lookup`, `$unwind`), де кожна стадія обробляє потік документів від попередньої.

Індекс – це B-дерево ключів одного або кількох полів із посиланнями на документи. Без придатного індексу планувальник виконує COLLSCAN (читання всієї колекції), з індексом – IXSCAN і FETCH лише потрібних

документів. Метод `explain("executionStats")` показує виграшний план і лічильники `totalDocsExamined`, `totalKeysExamined`, `executionTimeMillis`. Для складених індексів діє правило ESR: спочатку поля рівності, потім поле сортування, потім діапазон. Time-series колекції (у MongoDB починаючи з п'ятої версії) групують виміри у внутрішні бакети зі стовпцевим стисненням.

Результати виконання практичного заняття

1 Підготовка MongoDB

Через блокування `fastdl.mongodb.org` проксі сервер встановлено з conda-forge (micromamba), mongosh – з npm, PyMongo – через pip. Використано MongoDB Community Server 8.3.7 (рушій WiredTiger, стиснення snappy), mongosh 2.14.1, PyMongo 4.19.0, Python 3.13.18 і pandas 3.0.5; усі часи виміряно на машині з Intel Xeon 2,10 ГГц (4 vCPU) і ОЗП 15,7 ГіБ під Linux (glibc 2.39). Compass потребує дисплея, тому його операції виконано в mongosh. PyMongo підключається з `tz_aware=True`, щоб дати повертались в UTC (додаток А.1).

Під час повторних вимірів на машині паралельно працювали задачі Spark, і loadavg за 1 хв становив 5–12 при 4 vCPU (записано в CSV поруч із кожним виміром). Тому абсолютні часи мають розкид, і порівнюються лише варіанти одного прогону, виміряні з прогрівом і медіаною.

2 Створення бази даних і колекцій

База `bigdata_db` і звичайні колекції створюються неявно під час першого `insert_many`, а time-series колекцію створено явно до вставки методом `create_collection` з параметром `timeseries` (поле часу `ts`, поле метаданих `meta`, гранулярність `minutes`; додаток А.2). Створено сім колекцій. Їх призначення і розміри за `collStats` зведено в таблицю 1.

Таблиця 1 – Колекції бази bigdata_db (розміри в МіБ, 1 МіБ = 2^20 Б)

Колекція	Призначення	Документів	size	storageSize	Індекси
candles_1m	хвилинні свічки, плоска схема (основна)	7 816 537	894,53	383,05	280,07
candles_1m_ts	ті самі свічки у time-series колекції	7 816 537	76,94	69,31	0,45
candles_1h	годинні свічки, плоска схема	130 276	13,17	6,56	2,72
candles_1d	добові свічки	5 429	0,75	0,30	0,25
candles_1d_buckets	доба з масивом 24 годинних свічок	5 429	11,68	5,25	0,14
spark_daily_features	добові ознаки, обчислені Spark у П32	5 429	2,04	0,96	0,14
anomalies	аномальні хвилини П33 (правило v2)	5 437	1,59	0,63	0,25

Розміри знято після створення індексів розділу 6: для `candles_1m` це `_id_` (82,23), `ts_1` (90,43) і `volume_-1` (107,41), а time-series колекція має лише автоматичний індекс `meta_1_ts_1`. Кількість документів, перевірена `countDocuments` у mongosh, збігається з кількістю рядків джерел.

2.1 Обґрунтування схеми

Для хвилинних свічок порівняно три варіанти: плоский документ на хвилину (`candles_1m`), time-series колекцію (`candles_1m_ts`) і шаблон bucket, де один документ містить масив вимірів (`candles_1d_buckets`, доба з 24 годинами). Обсяг зберігання показано на рисунку 1. Розмір BSON плоскої колекції – 894,53 МіБ, або 937,98 МБ (120 Б на документ), у 2,36 раза більше за вихідний CSV (396,85 МБ, тобто 378,47 МіБ), бо кожен документ повторює імена полів. Після стиснення на диску лишається 383,05 МіБ. Time-series колекція займає 69,31 МіБ, у 5,5 раза менше, а її індекс `meta_1_ts_1` – 0,45 МіБ проти 90,43 МіБ для `ts_1` плоскої колекції. Сервер згрупував виміри в 7 817 бакетів і стиснув їх стовпцями. Шаблон bucket для годинних свічок дає менший виграш (5,25 проти 6,56 МіБ), але зменшує кількість документів у 24 рази.

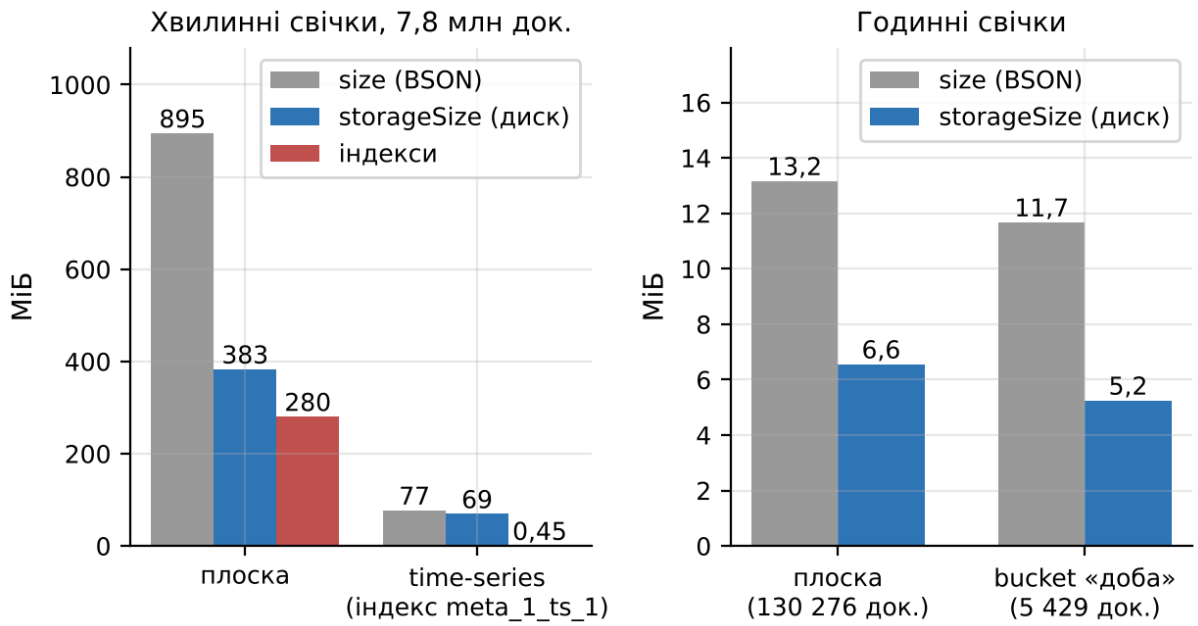


Рисунок 1 – Обсяг зберігання, МіБ: звичайна і time-series колекції хвилинних свічок, плоска і bucket-схема годинних свічок

За обсягом time-series колекція явно вигідніша. Щоб вибір схеми спирався на час запитів, а не лише на обсяг, три типові запити виконано на обох колекціях з прогрівом і медіаною з 5 запусків (рисунок 2).

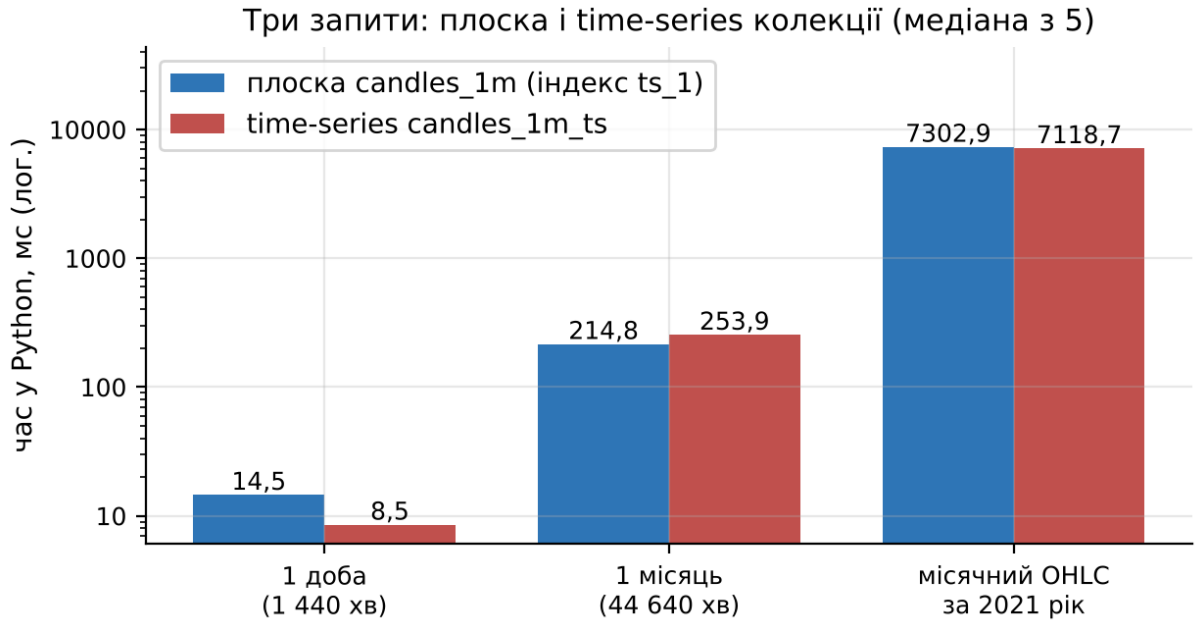


Рисунок 2 – Час запитів до плоскої і time-series колекцій (повний час у Python, медіана з 5 запусків)

Доба читається з time-series колекції швидше (8,5 проти 14,5 мс), місяць – повільніше (253,9 проти 214,8 мс), місячний OHLC за 2021 рік – за однаковий час (7 118,7 і 7 302,9 мс); розкид запусків перевищує різницю медіан. Основною обрано плоску колекцію `candles_1m`: вона дозволяє

вторинний індекс на будь-якому полі (`volume_-1`), оновлення окремих хвилин без обмежень, а `explain` рахує хвилини, а не бакети. Time-series колекція придатна для архіву.

3 Імпорт даних

3.1 Завантаження

Джерелом слугує оброблений набір `btc_1min.parquet` (час UTC) та похідні годинний і добовий набори. Документи формуються з DataFrame пакетами і вставляються методом `insert_many(ordered=False)`. Утиліту `mongoimport` не використано, бо типи Date та Int64 для CSV довелося б описувати вручну, а з DataFrame вони переходять у BSON напряму.

Повне завантаження 7 816 537 документів у `candles_1m` тривало 117,7 с: 18,2 с – побудова документів у Python, 99,6 с – власне вставка зі швидкістю 78,5 тис. док/с. Пакет 20 000 документів по 120 Б (близько 2,4 МБ) значно менший за межу повідомлення протоколу (48 МБ), тож драйвер його не дробить, а весь набір передається за 391 звернення; інші розміри пакета не вимірювалися. Time-series колекція `candles_1m_ts` завантажена за 99,2 с і приймає дані на 27 % швидше (вставка 78,5 с, 99 638 док/с), бо записує 7 817 стиснених бакетів.

3.2 Структура документів і типи полів

Структуру документів трьох схем показано на рисунку 3: плоский документ має `_id` і сім полів (`ts`, `unix` і OHLCV), time-series документ додає поле `meta`, а документ bucket-схеми містить вкладений масив `hours` (на рисунку виведено перший з 24 елементів).

```
$ mongosh --quiet --eval 'const P=(d)=>print(JSON.stringify(d)); P(db.candles_1m.findOne({
  }, {_id:0})); P(db.candles_1m_ts.findOne({}, {_id:0})); P(db.candles_1d_buckets.aggregat
  e([{$limit:1},{$project:{_id:0,date:1,n_hours:1,hours:{$slice:["$hours",1]}}}]).toArray(
  )[0])' bigdata_db
{"ts":"2012-01-01T00:01:00.000Z","unix":{"high":0,"low":1325376060,"unsigned":false},"open
↪ ":4.58,"high":4.58,"low":4.58,"close":4.58,"volume":0}
{"ts":"2012-01-01T00:01:00.000Z","meta":{"symbol":"BTCUSD"},"volume":0,"high":4.58,"unix":
↪ {"high":0,"low":1325376060,"unsigned":false},"low":4.58,"close":4.58,"open":4.58}
{"date":"2012-01-01T00:00:00.000Z","n_hours":24,"hours":[{"hour":0,"open":4.58,"high":4.58
↪ ,"low":4.58,"close":4.58,"volume":0}]}
```

Рисунок 3 – Перші документи колекцій candles_1m, candles_1m_ts і candles_1d_buckets (mongosh)

Поле `unix` типу Int64 `JSON.stringify` виводить як об'єкт. Типи полів визначено агрегацією `{$group: {_id: {$type: "$поле"}}}` по всіх документах кожної колекції; результат наведено в таблиці 2.

Таблиця 2 – BSON-типи полів

Колекція	Поля	BSON-тип
candles_1m, candles_1h	ts	date
candles_1m	unix	long
усі колекції свічок	open, high, low, close, volume	double
candles_1d, candles_1d_buckets	date / year, month, n_minutes, n_hours	date / int
candles_1d_buckets	hours	array вкладених документів
spark_daily_features	date / rv, hl_rel, ret_1d, vol_7d та ще 12 полів / n_minutes, n_flat, year, month, dow, is_complete	date / double / int
anomalies	ts, date / score, z_ret, z_vol, log_return та ще 7 полів / k, year, ts_epoch / rule, rule_version / in_stream_replay	date / double / int / string / bool

Кожне поле має єдиний тип у всіх документах, крім трьох полів Spark із типом null (п. 3.3). Час збережено як BSON Date, а не рядок, тому працюють діапазонні фільтри, оператори `$year`, `$month`, `$hour` і сортування в хронологічному порядку.

3.3 Пропущені значення

У колекціях свічок для кожного поля підраховано документи, де поле відсутнє, дорівнює null або NaN: у всіх 27 полях таких документів 0. У `spark_daily_features` значення NaN з Parquet замінено на null під час завантаження: 1 у `ret_1d` (перша доба не має попередньої), по 2 у `vol_7d` і `vol_30d` (вікна ковзного стандартного відхилення на початку історії). Хвилини з нульовим обсягом (1 312 419, тобто 16,79 %) не є пропусками: так джерело заповнює хвилини без угод, тому їх збережено.

4 Виконання запитів

Реалізовано вісім запитів `find`, які разом охоплюють пошук, фільтрацію, сортування, обмеження результатів і проєкцію (таблиця 3). Час одного запуску виміряно в Python разом із передачею результатів клієнту за наявності індексів з розділу 6.

Таблиця 3 – Запити до бази bigdata_db

№	Призначення	Колекція	Оператори	Документів	Час, мс
Q1	доби 2021 року з close > 60 000	candles_1d	$gte, $lt, $gt, проєкція, sort	39	4,7
Q2	топ-10 діб за обсягом	candles_1d	sort(volume: −1), limit(10)	10	4,0
Q3	5 найнижчих low 12.03.2020, 12:00–13:00 UTC	candles_1m	діапазон ts, sort, limit(5)	5	1,1
Q4	5 хвилин з найбільшим обсягом	candles_1m	$gt, sort, limit(5)	5	2,1
Q5	повні доби з ret_1d < −0,10	spark_daily_features	рівність, $lt, sort, limit(10)	10	4,3
Q6	неповні доби (n_minutes < 1 440)	candles_1d	$lt, проєкція	2	2,4
Q7	кількість хвилин з volume = 0	candles_1m	countDocuments	1	370,9
Q8	листопад–грудень 2022 з flat_share = 0	spark_daily_features	$and, $in	2	10,5

Запит Q3 вибирає з 7,8 млн документів одну годину обвалу 12.03.2020 і сортує її за мінімумом (рисунок 4).

```
$ mongosh --quiet --eval 'db.candles_1m.find({ts:{$gte:ISODate("2020-03-12T12:00:00Z"),$lt
  :ISODate("2020-03-12T13:00:00Z")}},{_id:0,ts:1,low:1,close:1,volume:1}).sort({low:1}).li
  mit(5).forEach(d => print(d.ts.toISOString(), String(d.low).padStart(8), String(d.close)
  .padStart(8), d.volume.toFixed(2).padStart(7)))' bigdata_db
2020-03-12T12:49:00.000Z     5850  5880.53   90.05
2020-03-12T12:52:00.000Z  5868.46  5868.46   76.24
2020-03-12T12:50:00.000Z  5874.02   5958.8   71.68
2020-03-12T12:54:00.000Z  5883.72  5924.93   66.28
2020-03-12T12:53:00.000Z  5887.69  5933.88  108.60
```

Рисунок 4 – Запит Q3: найнижчі ціни 12.03.2020 між 12:00 і 13:00 UTC

Мінімум цієї години – 5 850 USD о 12:49 UTC при обсязі 90,0 BTC за хвилину, а вибірка за індексом `ts_1` займає 1,1 мс. За Q5 найгірші повні доби в результатах Spark – 11.04.2013 (лог-дохідність −0,664) і 12.03.2020 (−0,494). Q6 знаходить дві неповні доби, першу (01.01.2012, 1 439 хвилин) і останню (11.11.2026, 218 хвилин). Q8 повертає 09.11 і 10.11.2022 – дні краху FTX. Найповільніший Q7 (370,9 мс) не читає документів: за `explain` це COUNT_SCAN, що перебирає 1 312 419 ключів індексу `volume_-1`.

5 Агрегування

Побудовано п'ять конвеєрів над свічками і ознаками Spark та конвеєр із `$lookup` над аномаліями (розділ 7). Склад стадій і час виконання наведено в таблиці 4.

Таблиця 4 – Конвеєри агрегації

Конвеєр	Колекція	Стадії	Рядків	Час, с
A1 місячні OHLC	candles_1m	$sort → $group ($first, $max, $min, $last, $sum) → $sort → $project	179	103,88
A2 обсяг за годиною доби, 2018+	candles_1m	$match → $group ($hour) → $sort → $project	24	65,93
A3 річна волатильність	spark_daily_features	$match → $group → $sort → $project	15	0,325
A4 розподіл діб за ціною	candles_1d	$bucket	7	0,290
A5 річний обсяг	candles_1d	$group → $sort	15	0,090
ANOM аномалії і ознаки Spark	anomalies	$group → $lookup → $unwind → $project → $sort	1 572	0,192

Для A1 наведено медіану з 3 запусків, для решти – один запуск. Конвеєри над 7,8 млн хвилин тривають хвилину і більше, над добовими колекціями – десяті частки секунди.

5.1 Місячні OHLC

Конвеєр A1 ресемплює хвилинні свічки в місячні за правилом first/max/min/last/sum. Стадія `$sort` стоїть перед `$group`, бо без неї `$first` і `$last` брали б документи в порядку зберігання, який не гарантує хронології. Виконання конвеєра в mongosh для перших місяців 2021 року показано на рисунку 5.

```
$ mongosh --quiet --eval 'db.candles_1m.aggregate([{$sort:{ts:1}},{$group:{_id:{y:{$year:"
  $ts"},m:{$month:"$ts"}},open:{$first:"$open"},high:{$max:"$high"},low:{$min:"$low"},clos
  e:{$last:"$close"},volume:{$sum:"$volume"}}},{$sort:{_id:1}},{$match:{"_id.y":2021,"_id.
  m":{$lte:4}}}],{allowDiskUse:true}).forEach(d => print(d._id.y + "-" + String(d._id.m).p
  adStart(2,"0"), d.open, d.high, d.low, d.close, Math.round(d.volume)))' bigdata_db
2021-01 28999.63 42000 27734 33141.38 457461
2021-02 33110.32 58354.14 32333.01 45240.96 240351
2021-03 45261.56 61781.83 45020.25 58782.58 157899
2021-04 58789.96 64895.22 47004.2 57775.27 139624
```

Рисунок 5 – Конвеєр A1 у mongosh: місячні OHLC і обсяг, січень–квітень 2021 року

Коректність перевірено порівнянням з ресемплінгом pandas добових свічок: для всіх 179 місяців максимальна абсолютна різниця open, high, low і close дорівнює 0, а відносна різниця обсягу – $2{,}1 \cdot 10^{-16}$ (похибка округлення суми з плаваючою комою).

5.2 Обсяг за годиною доби

Конвеєр A2 групує хвилини з 2018 року за `$hour` (UTC). Період обмежено 2018 роком, бо раніше частка хвилин без угод була великою і

спотворювала середні. Результат подано на рисунку 6: найактивніша година – 14:00 UTC (4,96 BTC за хвилину), найспокійніша – 03:00 UTC (2,24 BTC), різниця в 2,2 раза.



Рисунок 6 – Середній обсяг за хвилину за годиною доби (UTC), 2018–2026 роки

Пік 13:00–16:00 UTC збігається з відкриттям фондового ринку США, отже, цілодобовий ринок біткоїна має виражену денну сезонність.

5.3 Річна волатильність і розподіл цін

Конвеєр A3 рахує річні показники за ознаками Spark лише для повних діб (`is_complete = 1`). Середня реалізована волатильність `rv` знизилася з 0,1152 у 2013 році до 0,0200 у 2023 році і з того часу не перевищувала 0,0260. Конвеєр `$bucket` (A4) розподіляє доби за ціною закриття: 1 332 доби в діапазоні 100–1 000 USD, 1 317 – у 10 000–50 000 USD і 217 діб понад 100 000 USD, перша з яких – 08.12.2024.

6 Індексація

Індекси створено для полів, за якими фільтрують і сортують запити (таблиця 5), а також `date_1` у `candles_1d` і `spark_daily_features` та `year_1` в `anomalies`. Для фази «до» видаляються всі вторинні індекси, для фази «після» створюється рівно один; після прогріву запит виконується 5 разів, і в таблицю 5 потрапляють медіани двох метрик: `executionTimeMillis` з `explain("executionStats")` (час на сервері) і

повний час виконання в Python разом з передачею результату. План «до» в усіх випадках – COLLSCAN (з SORT, крім E5b), а кількість повернутих документів до і після однакова (для E1 – 1 440, для E2 – 44 640). План «після» – IXSCAN і FETCH (для E3 ще LIMIT, для ANOM1 ще SORT), а кількість переглянутих ключів дорівнює кількості документів, крім ANOM1 (338 ключів на 20 документів).

Зміну плану для запиту E1 в mongosh показано на рисунку 7: «до» отримано примусовим повним скануванням `hint({$natural: 1})`, «після» – планом за замовчуванням.

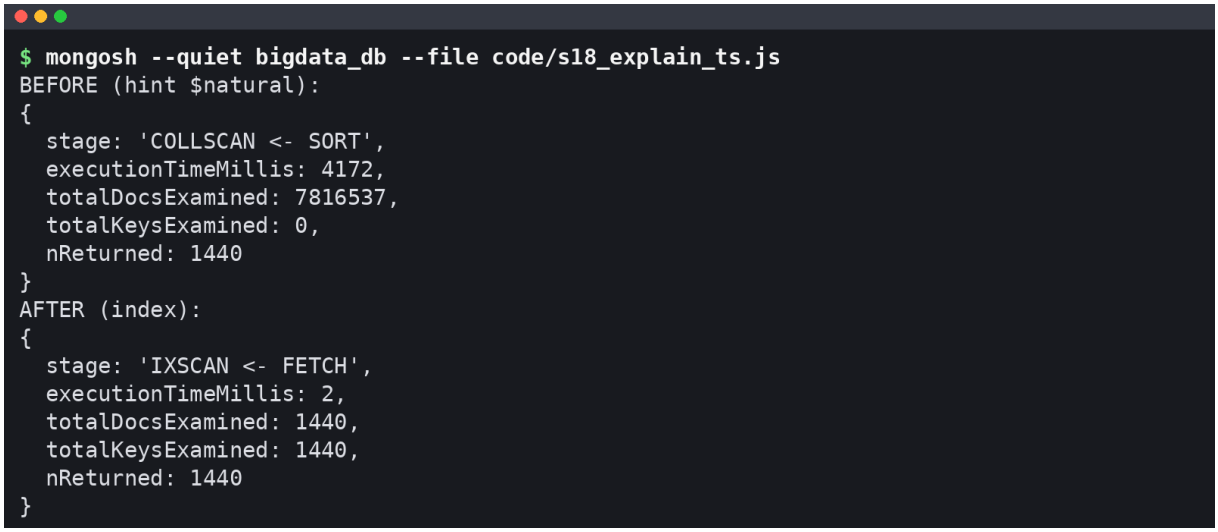


Рисунок 7 – План запиту E1 до (COLLSCAN) і після (IXSCAN) створення індексу ts_1

На скриншоті – один запуск, тому час відрізняється від медіани. Результати для всіх запитів наведено в таблиці 5: E1–E3 виконано на `candles_1m`, E4 – на `candles_1d`, E5a – на `candles_1h`, E5b – на `candles_1d_buckets`, ANOM1 і ANOM2 – на `anomalies`.

Таблиця 5 – Виконання запитів до і після створення індексу (медіана з 5 прогрітих запусків)

Запит	Індекс	explain, мс	Python, мс	Переглянуто док.
E1 доба 19.05.2021	ts_1	4 637 → 1	4 548,3 → 5,5	7 816 537 → 1 440
E2 травень 2021	ts_1	5 886 → 50	4 921,9 → 191,2	7 816 537 → 44 640
E3 топ-10 за volume	volume_-1	5 399 → 0	3 771,6 → 0,5	7 816 537 → 10
E4 2021, volume > 9 483,33	year_1_volume_-1	2 → 0	3,0 → 0,6	5 429 → 36
E5a 24 години	ts_1	86 → 0	73,5 → 0,6	130 276 → 24
E5b доба-bucket	date_1	2 → 0	3,9 → 0,5	5 429 → 1
ANOM1 2021, топ-20 за score	date_1_score_-1	3 → 0	3,6 → 1,0	5 437 → 20
ANOM2 доба 12.03.2020	date_1_score_-1	2 → 0	3,2 → 0,9	5 437 → 82

Час на сервері для всіх запитів подано на рисунку 8 у логарифмічній шкалі.

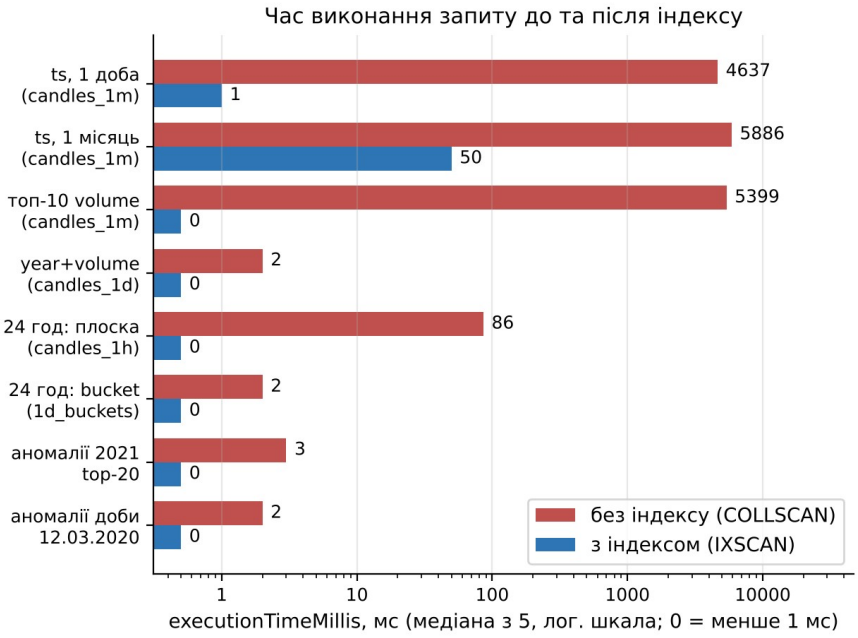


Рисунок 8 – Час виконання запитів на сервері (executionTimeMillis) без індексу і з індексом, медіана з 5 запусків

На хвилинній колекції індекс `ts_1` скорочує вибірку доби з 4 637 до 1 мс за `executionTimeMillis`, а за повним часом у Python – з 4 548,3 до 5,5 мс (у 827 разів); топ-10 за обсягом прискорюється з 3 771,6 до 0,5 мс. Для травня 2021 року виграш менший (у 118 разів на сервері і в 26 разів у Python), бо час займають FETCH і передача 44 640 документів. Складений індекс `{year: 1, volume: -1}` відповідає правилу ESR, тому стадія SORT зникає. В `anomalies` для ANOM1 стадія SORT лишилася: діапазон

дат охоплює 338 ключів, а порядок `score` індекс гарантує лише в межах однієї дати; для рівності на дату (ANOM2) сортування не потрібне. На малих колекціях виграш за повним часом становить лише 2,3–3,4 мс.

Ціна індексів – місце, час побудови і уповільнення запису. Індекс `ts_1` займає 90,43 МіБ і будується 26,26 с, `volume_-1` – 107,41 МіБ і 31,29 с; разом з `_id` індекси `candles_1m` займають 280,07 МіБ при 383,05 МіБ даних. Для оцінки вартості запису 200 000 хвилинних документів вставлено пакетами по 20 000 у копію колекції без вторинних індексів і з `ts_1` та `volume_-1` (медіана з 3 запусків): пропускна здатність падає з 120 458 до 66 824 док/с, тобто вставка повільніша на 80,3 %. Побудова обох індексів після вставки займає 1,183 с, тому історію вигідніше завантажувати без вторинних індексів. Без індексу bucket-схема (E5b) повертає добу за 3,9 мс проти 73,5 мс для плоскої (E5a), з індексами обидві – за 0,5–0,6 мс.

6.1 Агрегація, яка не прискорюється індексом

Конвеєр A1 виконано по 3 рази з індексом `ts_1` (медіана 103,88 с) і без нього з `hint({$natural: 1})` (104,08 с). Різниця 0,2 % менша за розкид окремих запусків (від 97,34 до 104,53 с), тож індекс фактично не допомагає. За `explain` обидва плани переглядають усі 7 816 537 документів; без індексу на диск виливається SORT (8 скидань), але в обох планах з диском працює і GROUP (3 908 268 скидань), що й визначає час. Покривний індекс на `{ts, open, high, low, close, volume}` прибрав би FETCH, але не групування; його не перевірено. Місячні OHLC з `candles_1d` (5 429 документів) обчислюються за 0,108 с, тому місячні звіти доцільно рахувати з добових агрегатів.

7 Інтеграція з результатами Spark і ПЗ3

Колекцію `spark_daily_features` (5 429 документів, 24 поля) завантажено з добових ознак повторного прогону ПЗ2 (Spark). Порівняно з попереднім прогоном відрізняються лише `rv`, `std_ret_1m` і `vol60_mean` у 137–178 рядках (для `rv` не більше ніж на 0,00065), що не змінило чисел

розділів 4, 5.3 і 7. Колекцію `anomalies` завантажено з результату ПЗ3 `pz3_anomalies.csv` (5 437 документів, 0,16 с). Правило v2 з ПЗ3 позначає хвилину аномальною, якщо модуль її лог-дохідності перевищує 9σ, де σ = $Q_{0,90}(|r|)/1{,}6449$ за активними хвилинами попередніх 7 діб, а обсяг вищий за тижневу медіану. Отримано 5 437 аномальних хвилин у 1 572 добах.

Результат конвеєра з `$lookup`, що з'єднує кожну добу з аномаліями з її ознаками Spark за полем `date`, показано на рисунку 9.

```
$ mongosh --quiet --eval 'db.anomalies.aggregate([{$group:{_id:"$date",n_anom:{$sum:1},max
  _score:{$max:"$score"}}},{$lookup:{from:"spark_daily_features",localField:"_id",foreignF
  ield:"date",as:"f"}},{$unwind:"$f"},{$project:{_id:0,date:"$_id",n_anom:1,max_score:1,rv
  :"$f.rv",volume:"$f.volume"}},{$sort:{n_anom:-1}},{$limit:6}]).forEach(d => print(d.date
  .toISOString().slice(0,10), String(d.n_anom).padStart(4), d.max_score.toFixed(2).padStar
  t(6), d.rv.toFixed(4), Math.round(d.volume)))' bigdata_db
2020-03-12   82   5.85 0.2111 58513
2020-03-13   75   3.47 0.2784 54420
2019-04-02   66   4.60 0.0758 30686
2022-11-08   59   9.48 0.0933 12036
2018-11-14   58   4.86 0.0699 25122
2013-04-10   55  10.22 0.8632 20889
```

Рисунок 9 – Конвеєр $lookup: доби з найбільшою кількістю аномалій та їх ознаки Spark

Агрегація за роками дає найбільше аномалій у 2019 (777) і 2023 (749) роках. Перекіс у бік тонкого ринку перевірено за частками з ПЗ3: у 2012–2016 роках позначено 0,058 % оцінених хвилин, у 2018+ – 0,096 %. Виняток – 2013 рік (461 аномалія, 0,154 %), на який припадають три дати з топ-10: 10.04, 02.10 і 19.11.2013. У `$lookup` пару знайшли всі 1 572 доби. Перше місце посідає 12.03.2020 (82 аномалії, `rv` 0,211, лог-дохідність −0,494), друге – 13.03.2020 (75), четверте – 08.11.2022 (59, найбільший score 9,48 серед перших п'яти діб, крах FTX). Порівняння діб з аномаліями з рештою повних діб наведено в таблиці 6.

Таблиця 6 – Ознаки Spark для діб з аномаліями і решти повних діб

Група	Діб	Середнє rv	Медіана rv	Середнє hl_rel	Медіана обсягу, BTC
доби з аномаліями	1 572	0,0506	0,0353	0,0695	4 853,7
решта повних діб	3 855	0,0453	0,0356	0,0458	3 798,2

Медіана `rv` в обох групах майже однакова (0,0353 і 0,0356), а середній відносний діапазон `hl_rel` і медіанний обсяг у добах з аномаліями вищі в 1,5 і 1,3 раза. Правило ПЗ3 знаходить раптові хвилинні

стрибки з підвищеним обсягом, які розширюють денний діапазон, але не роблять увесь день волатильним.

8 Аналіз можливостей MongoDB для великих даних

Документна модель дозволила зберегти в одній базі свічки, агрегати, ознаки Spark і аномалії з різною структурою без міграцій. Сервер приймав 78,5 тис. док/с у звичайну і 99,6 тис. у time-series колекцію. Індекси переводять селективні запити з 4–6 с до мілісекунд, а Aggregation Framework достатній для ресемплінгу OHLC і `$lookup`.

Обмеження виявилися на повних проходах. Агрегація по всіх хвилинах триває 66–104 с, індекс її не прискорює, а без шардування конвеєр не розподіляється між вузлами. Для порівняння, у П32 на тій самій машині Spark local[*] будував добові OHLC з усіх 7,8 млн хвилин за 4,547 с (медіана з 3 запусків разом з читанням CSV), pandas – за 4,884 с, а Spark із закешованим DataFrame – за 0,618 с. Умови відрізняються (добові, а не місячні групи; читання CSV, а не BSON; loadavg 8,6 на початку прогону П32 проти 5–7 під час А1), проте різниця з А1 (103,88 с) перевищує порядок, і навантаженням її не пояснити. Тому MongoDB доцільно використовувати як шар зберігання і швидкого доступу: перерахунки всієї історії виконувати в Spark, а в MongoDB завантажувати їх результати (добові агрегати, ознаки, аномалії) з індексами під типові запити.

Висновки

1. Встановлено MongoDB Community Server 8.3.7, mongosh 2.14.1 і PyMongo 4.19.0; Compass у контейнері без дисплея недоступний, його операції виконано в mongosh.
2. Створено базу `bigdata_db` із семи колекцій; time-series колекція займає 69,31 МіБ проти 383,05 МіБ для плоскої, але за часом запитів схеми рівноцінні, і основною обрано плоску.
3. Імпортовано 7 816 537 хвилинних документів за 117,7 с (вставка 78,5 тис. док/с, пакети по 20 000); визначено структуру, BSON-типи і

пропуски: у 27 полях свічок їх немає, в ознаках Spark 5 значень null, нульовий обсяг 16,79 % хвилин є заповненням сітки.

4. Вісім запитів охоплюють пошук, фільтрацію, сортування, обмеження і проєкцію; усі, крім Q7 (COUNT_SCAN, 370,9 мс), виконуються за 1,1–10,5 мс.
5. Шість конвеєрів агрегації дали місячні OHLC, що збігаються з pandas для 179 місяців, денну сезонність обсягу (пік о 14:00 UTC) і зниження середньої `rv` з 0,1152 до 0,0200.
6. Індекс `ts_1` скоротив вибірку доби з 4 637 до 1 мс на сервері (з 4 548,3 до 5,5 мс у Python) і з 7 816 537 до 1 440 переглянутих документів, але разом з `volume_-1` уповільнив запис на 80,3 %; повна агрегація A1 від індексу не виграє (103,88 проти 104,08 с).
7. Імпортовано результати П32 (5 429 діб ознак Spark) і П33 (5 437 аномалій, k = 9) і з'єднано їх через `$lookup`.
8. MongoDB ефективна як сховище результатів обробки з мілісекундним доступом за індексами, а повні проходи доцільно виконувати в Spark (добові OHLC за 4,547 с проти 103,88 с для місячних OHLC у MongoDB).

У результаті отримано практичні навички створення документного сховища, виконання запитів, агрегування великих наборів даних і використання індексації.

Додаток А. Лістинг програми

А.1 Файл pz4_common.py – спільні утиліти: підключення, запис результатів, вимірювання до/після індексу

```
# -*- coding: utf-8 -*-
"""Спільні утиліти ПЗ4: підключення до MongoDB, шляхи, запис результатів."""
import os, csv, json, time, statistics, datetime
from bson import json_util
from pymongo import MongoClient
from bson.codec_options import CodecOptions

BASE = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
W = os.path.abspath(os.path.join(BASE, ".."))
RES, FIG, EXPL = f"{BASE}/results", f"{BASE}/figures", f"{BASE}/results/explain"
for d in (RES, FIG, EXPL):
    os.makedirs(d, exist_ok=True)
DB_NAME = "bigdata_db"
URI = "mongodb://127.0.0.1:27017"


def get_db(tz_aware=True):
    """Повертає базу bigdata_db; дати читаються як UTC-aware."""
    cl = MongoClient(URI)
    return cl.get_database(DB_NAME, codec_options=CodecOptions(tz_aware=tz_aware,
   tzinfo=datetime.timezone.utc))


def save_csv(name, rows, header=None):
    """Зберігає список словників/кортежів у results/<name>.csv."""
    path = f"{RES}/{name}.csv"
    with open(path, "w", newline="", encoding="utf-8") as f:
        if rows and isinstance(rows[0], dict):
            header = header or list(rows[0].keys())
            w = csv.DictWriter(f, fieldnames=header); w.writeheader();
  w.writerows(rows)
        else:
            w = csv.writer(f)
            if header: w.writerow(header)
            w.writerows(rows)
    return path


def append_timing(stage, seconds, note=""):
    """Дописує вимір часу до results/timings.csv."""
    path = f"{RES}/timings.csv"
    new = not os.path.exists(path)
    with open(path, "a", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        if new: w.writerow(["stage", "seconds", "note"])
        w.writerow([stage, f"{seconds:.3f}", note])


def explain_find(db, coll, flt, sort=None, limit=0, proj=None, hint=None):
    """explain('executionStats') для find."""
    cmd = {"find": coll, "filter": flt}
    if sort: cmd["sort"] = dict(sort)
    if limit: cmd["limit"] = limit
    if proj: cmd["projection"] = proj
    if hint is not None: cmd["hint"] = hint
    return db.command({"explain": cmd, "verbosity": "executionStats"})


def stages(plan, out=None):
    """Збирає назви стадій виграшного плану (від кореня до листка)."""
    out = [] if out is None else out
    out.append(plan["stage"])
    if "inputStage" in plan: stages(plan["inputStage"], out)
```

```
    for s in plan.get("inputStages", []): stages(s, out)
    return out


def summarize(ex):
    """Витягує ключові метрики з виводу explain."""
    es = ex["executionStats"]
    return dict(executionTimeMillis=es["executionTimeMillis"],
   totalDocsExamined=es["totalDocsExamined"],
                totalKeysExamined=es["totalKeysExamined"],
   nReturned=es["nReturned"],
                winning_plan=" > ".join(stages(ex["queryPlanner"]
   ["winningPlan"])))


# ---------- порівняння до/після індексу (спільне для pz4_indexes.py та
   pz4_anomalies.py) ----------
REPS = 5
_db = get_db()


def drop_secondary(coll):
    """Видаляє всі індекси колекції, крім _id_."""
    for ix in _db[coll].index_information():
        if ix != "_id_": _db[coll].drop_index(ix)


def measure(tag, coll, flt, sort, limit, hint=None):
    """warm-up + 5 вимірювань: executionTimeMillis (з explain) та wall-clock
   фактичного виконання."""
    run = lambda: list(_db[coll].find(flt, sort=sort, limit=limit))
    load_start = os.getloadavg()[0]
    explain_find(_db, coll, flt, sort, limit)                # прогрів кешу
    ex_t, wall = [], []
    for _ in range(REPS):
        ex = explain_find(_db, coll, flt, sort, limit)
        ex_t.append(ex["executionStats"]["executionTimeMillis"])
        t = time.perf_counter(); run(); wall.append((time.perf_counter() - t) *
   1000)
    s = summarize(ex)
    json.dump(json.loads(json_util.dumps(ex)), open(f"{EXPL}/{tag}.json", "w"),
   indent=1)
    s.update(loadavg_1m=f"{load_start:.2f}->{os.getloadavg()[0]:.2f}",
   explain_ms_runs=ex_t, wall_ms_runs=[round(x, 1) for x in wall],
             explain_ms_median=statistics.median(ex_t),
   wall_ms_median=round(statistics.median(wall), 1))
    return s


def experiment(name, coll, flt, sort, limit, spec, descr):
    drop_secondary(coll)
    before = measure(f"{name}_before", coll, flt, sort, limit)
    t = time.perf_counter(); iname = _db[coll].create_index(spec); build =
   time.perf_counter() - t
    after = measure(f"{name}_after", coll, flt, sort, limit)
    st = _db.command("collStats", coll)
    isize = st["indexSizes"][iname] / 2**20   # МіБ
    out = []
    for phase, m in (("до", before), ("після", after)):
        out.append(dict(experiment=name, description=descr, collection=coll,
   phase=phase, index=iname if phase == "після" else "-",

   executionTimeMillis_explain_median=m["explain_ms_median"],
   wall_ms_median=m["wall_ms_median"],
                        totalDocsExamined=m["totalDocsExamined"],
   totalKeysExamined=m["totalKeysExamined"],
                        nReturned=m["nReturned"], winning_plan=m["winning_plan"],
   loadavg_1m=m["loadavg_1m"],
                        explain_ms_runs=m["explain_ms_runs"],
   wall_ms_runs=m["wall_ms_runs"],
                        index_size_MiB=round(isize, 2) if phase == "після" else
```

```
  0, index_build_s=round(build, 2) if phase == "після" else 0))
   for o in out: print(o["experiment"], o["phase"],
  o["executionTimeMillis_explain_median"], o["wall_ms_median"],
                       o["totalDocsExamined"], o["totalKeysExamined"],
  o["nReturned"], o["winning_plan"], flush=True)
   return out
```

A.2 Файл pz4_load.py – створення колекцій та імпорт даних

```
# -*- coding: utf-8 -*-
"""Завдання 2-3, 7: створення бази bigdata_db, колекцій та імпорт даних.
Колекції: candles_1m (повні 7,8 млн), candles_1m_ts (time-series), candles_1h,
  candles_1d,
candles_1d_buckets (bucket pattern), spark_daily_features (результати ПЗ2).
Заміряється пропускна здатність insert_many."""
import sys, time, datetime, math
import numpy as np, pandas as pd
from bson.int64 import Int64
from pymongo import MongoClient
sys.path.insert(0, __import__("os").path.dirname(__file__))
from pz4_common import *

SPARK_ONLY = "--only-spark" in sys.argv   # перезавантажити лише
  spark_daily_features
DATA = f"{W}/data"
BATCH = 20000
db = get_db()
log = []


def utc_list(series):
    """Конвертація datetime64 (UTC) у список datetime з tz=UTC."""
    return list(pd.DatetimeIndex(series).tz_convert("UTC").to_pydatetime()) if
  getattr(series.dt, "tz", None) \
        else list(pd.DatetimeIndex(series).tz_localize("UTC").to_pydatetime())


def load_1m(name, ts_mode=False, limit=None):
    """Імпорт хвилинних свічок батчами; повертає (рядків, сек. побудови, сек.
  вставки)."""
    df = pd.read_parquet(f"{DATA}/btc_1min.parquet")
    if limit: df = df.iloc[:limit]
    db.drop_collection(name)
    if ts_mode:
        db.create_collection(name, timeseries={"timeField": "ts", "metaField":
  "meta", "granularity": "minutes"})
    coll = db[name]
    n = len(df); t_build = t_ins = 0.0
    for s in range(0, n, BATCH):
        c = df.iloc[s:s + BATCH]
        t0 = time.perf_counter()
        ts = utc_list(c["datetime_utc"]); un = c["Timestamp"].tolist()
        o, h, l, cl, v = (c[k].tolist() for k in ("Open", "High", "Low", "Close",
  "Volume"))
        if ts_mode:
            docs = [{"ts": ts[i], "meta": {"symbol": "BTCUSD"}, "unix":
  Int64(un[i]), "open": o[i], "high": h[i],
                     "low": l[i], "close": cl[i], "volume": v[i]} for i in
  range(len(c))]
        else:
            docs = [{"ts": ts[i], "unix": Int64(un[i]), "open": o[i], "high":
  h[i], "low": l[i],
                     "close": cl[i], "volume": v[i]} for i in range(len(c))]
        t1 = time.perf_counter()
        coll.insert_many(docs, ordered=False)
        t2 = time.perf_counter()
        t_build += t1 - t0; t_ins += t2 - t1
        if (s // BATCH) % 40 == 0: print(name, s, n, f"{t_ins:.0f}s", flush=True)
    return n, t_build, t_ins
```

```
def clean(v):
    """NaN -> None (пропущене значення в MongoDB зберігається як null)."""
    return None if isinstance(v, float) and math.isnan(v) else v


if not SPARK_ONLY:
    # ---------- candles_1m ----------
    for name, ts_mode in (("candles_1m", False), ("candles_1m_ts", True)):
        n, tb, ti = load_1m(name, ts_mode)
        log.append(dict(collection=name, docs=n, batch_size=BATCH,
  build_docs_s=round(tb, 2), insert_s=round(ti, 2),
                        insert_docs_per_s=round(n / ti), total_s=round(tb + ti,
  2)))
        append_timing(f"load_{name}", tb + ti, f"{n} docs, insert only {ti:.1f}s,
  {n/ti:.0f} docs/s")
        print(log[-1], flush=True)

    # ---------- candles_1h, candles_1d, candles_1d_buckets ----------
    h = pd.read_parquet(f"{DATA}/btc_1h.parquet"); d =
  pd.read_parquet(f"{DATA}/btc_1d.parquet")
    t0 = time.perf_counter()
    db.drop_collection("candles_1h"); db.drop_collection("candles_1d");
  db.drop_collection("candles_1d_buckets")
    tsh = utc_list(h["datetime_utc"])
    db.candles_1h.insert_many([{"ts": tsh[i], "open": r.Open, "high": r.High,
  "low": r.Low, "close": r.Close,
                               "volume": r.Volume} for i, r in
  enumerate(h.itertuples())], ordered=False)
    tsd = utc_list(d["datetime_utc"])
    db.candles_1d.insert_many([{"date": tsd[i], "year": tsd[i].year, "month":
  tsd[i].month, "open": r.Open, "high": r.High,
                               "low": r.Low, "close": r.Close, "volume":
  r.Volume, "n_minutes": r.n_minutes}
                              for i, r in enumerate(d.itertuples())],
  ordered=False)
    # bucket pattern: один документ = доба з вкладеним масивом 24 годинних свічок
    h["day"] = h["datetime_utc"].dt.floor("D")
    buckets = []
    for day, g in h.groupby("day"):
        dd = pd.Timestamp(day).tz_convert("UTC").to_pydatetime() if
  pd.Timestamp(day).tzinfo else
  pd.Timestamp(day).tz_localize("UTC").to_pydatetime()
        buckets.append({"date": dd, "year": dd.year, "month": dd.month,
  "n_hours": len(g),
                        "hours": [{"hour": int(x.datetime_utc.hour), "open":
  x.Open, "high": x.High, "low": x.Low,
                                   "close": x.Close, "volume": x.Volume} for x in
  g.itertuples()]})
    db.candles_1d_buckets.insert_many(buckets, ordered=False)

# ---------- spark_daily_features (П32) ----------
f = pd.read_parquet(f"{W}/pz2/results/pz2_daily_features.parquet")
nan_counts = {c: int(f[c].isna().sum()) for c in f.columns if f[c].isna().any()}
db.drop_collection("spark_daily_features")
docs = []
for r in f.to_dict("records"):
    doc = {k: clean(v) for k, v in r.items()}
    doc["date"] = r["date"].to_pydatetime().replace(tzinfo=datetime.timezone.utc)
    for k in ("year", "month", "dow", "is_complete"): doc[k] = int(r[k])
    doc["n_minutes"] = int(r["n_minutes"]); doc["n_flat"] = int(r["n_flat"])
    docs.append(doc)
db.spark_daily_features.insert_many(docs, ordered=False)
if not SPARK_ONLY:
    append_timing("load_small_collections", time.perf_counter() - t0,
  "candles_1h, candles_1d, buckets, spark_daily_features")
    save_csv("pz4_load_throughput", log)
save_csv("pz4_spark_nan_to_null", [dict(field=k, nan_replaced_by_null=v) for k, v
  in nan_counts.items()])
print("DONE", {c: db[c].estimated_document_count() for c in
  db.list_collection_names()})
```

А.3 Файл pz4_schema.py – кількість документів, BSON-типи, пропуски, розміри колекцій

```
# -*- coding: utf-8 -*-
"""Завдання 3: кількість документів, структура, BSON-типи полів, пропущені
   значення, розміри колекцій."""
import json, sys
from bson import json_util
sys.path.insert(0, __import__("os").path.dirname(__file__))
from pz4_common import *

db = get_db()
colls = ["candles_1m", "candles_1m_ts", "candles_1h", "candles_1d",
   "candles_1d_buckets", "spark_daily_features"]
# 1. кількість документів і розміри (collStats)
rows = []
for c in colls:
    s = db.command("collStats", c)
    rows.append(dict(collection=c, count=db[c].count_documents({}) if c ==
   "candles_1m_ts" else s["count"], avgObjSize_B=s.get("avgObjSize", 0),
   size_MiB=round(s["size"] / 2**20, 2),
                    storageSize_MiB=round(s["storageSize"] / 2**20, 2),
   totalIndexSize_MiB=round(s["totalIndexSize"] / 2**20, 2),
                    nindexes=s["nindexes"]))
save_csv("pz4_storage", rows)
for r in rows: print(r)

# 2. BSON-типи кожного поля ($type) та пропущені значення (відсутнє / null / NaN)
types, miss = [], []
for c in colls:
    if c == "candles_1m_ts":
        continue
    fields = [k for k in db[c].find_one().keys() if k != "_id"]
    n = db[c].estimated_document_count()
    for f in fields:
        for r in db[c].aggregate([{"$group": {"_id": {"$type": "$" + f}, "n":
   {"$sum": 1}}}], allowDiskUse=True):
            types.append(dict(collection=c, field=f, bson_type=r["_id"],
   docs=r["n"]))
    grp = {"_id": None}
    for f in fields:
        grp[f] = {"$sum": {"$cond": [{"$or": [{"$eq": [{"$type": "$" + f},
   "missing"]}, {"$eq": ["$" + f, None]},
                                                {"$eq": ["$" + f,
   float("nan")]}]}, 1, 0]}}
    r = next(db[c].aggregate([{"$group": grp}], allowDiskUse=True))
    for f in fields:
        miss.append(dict(collection=c, field=f, missing_or_null_or_nan=r[f],
   total_docs=n))
save_csv("pz4_field_types", types)
save_csv("pz4_missing_values", miss)

# 3. приклади документів (Extended JSON, дати як $date)
ex = {}
for c in colls:
    d = db[c].find_one({}, sort=[("_id", 1)]) if c != "candles_1d_buckets" else
   db[c].find_one()
    if c == "candles_1d_buckets": d["hours"] = d["hours"][:2]   # скорочено до
   перших двох годин (решта опущена у звіті)
    ex[c] = json.loads(json_util.dumps(d,
   json_options=json_util.RELAXED_JSON_OPTIONS))
json.dump(ex, open(f"{RES}/pz4_sample_documents.json", "w"), ensure_ascii=False,
   indent=2)
print(json.dumps(ex["candles_1m"], indent=1))

# 4. порівняння розмірів для time-series: звичайна vs timeseries
ts = db.command("collStats", "candles_1m_ts")
json.dump(json.loads(json_util.dumps(ts)),
   open(f"{RES}/explain/collStats_candles_1m_ts.json", "w"), indent=1)
```

```
print({k: ts.get(k) for k in ("count", "size", "storageSize", "totalIndexSize")},
   ts.get("timeseries"))
```

A.4 Файл pz4_queries.py – запити find

```
# -*- coding: utf-8 -*-
"""Завдання 4: запити find (пошук, фільтрація, сортування, limit, projection).
   Результати -> results/pz4_q*.csv"""
import sys, time, datetime as dt
sys.path.insert(0, __import__("os").path.dirname(__file__))
from pz4_common import *

# підключення до бази bigdata_db (параметри в pz4_common.py)
db = get_db()
# допоміжна функція: UTC-дата для фільтрів по полях типу BSON Date
U = lambda *a: dt.datetime(*a, tzinfo=dt.timezone.utc)
# зведення за всіма запитами (id, опис, колекція, кількість рядків, час) ->
   pz4_queries_summary.csv
summary = []


def run(qid, descr, coll, flt, proj=None, sort=None, limit=0, count_only=False):
    # run(): виконує один find-запит (або count) і вимірює повний wall-clock час
   разом із передачею результату
    t = time.perf_counter()
    if count_only:
        # лише кількість документів за фільтром (count_documents), без повернення
   самих документів
        res = [{"count": db[coll].count_documents(flt)}]
    else:
        # find(фільтр, проєкція, сортування, ліміт): проєкція обмежує поля, sort
   і limit виконує сервер
        res = list(db[coll].find(flt, proj, sort=sort, limit=limit))
    # час у мілісекундах
    dtm = (time.perf_counter() - t) * 1000
    # приводимо дати до рядка, прибираємо службове поле _id, щоб результат був
   придатний для CSV
    rows = [{k: (v.strftime("%Y-%m-%d %H:%M") if isinstance(v, dt.datetime) else
   v) for k, v in r.items() if k != "_id"} for r in res]
    save_csv(f"pz4_{qid}", rows)  # повний результат запиту ->
   results/pz4_<qid>.csv
    # запис у зведену таблицю
    summary.append(dict(query=qid, description=descr, collection=coll,
   returned=len(res), wall_ms=round(dtm, 1)))
    # короткий вивід у консоль: id, опис, кількість рядків, час і перші 3 рядки
    print(qid, descr, len(res), f"{dtm:.0f} мс"); print(rows[:3])


# Q1: фільтр за діапазоном дат ($gte/$lt) + умова close>60000 ($gt), проєкція
   трьох полів, сортування за датою за зростанням;
# запит по індексу date_1 колекції добових свічок
run("q1_close_gt_60000_2021", "дні 2021 із close>60000 (сортування за датою)",
   "candles_1d",
    {"date": {"$gte": U(2021, 1, 1), "$lt": U(2022, 1, 1)}, "close": {"$gt":
   60000}}, {"_id": 0, "date": 1, "close": 1, "volume": 1}, [("date", 1)])
# Q2: сортування за спаданням обсягу + limit 10 -> топ-10 діб (використовує
   індекс year_1_volume_-1 / сортування за volume)
run("q2_top10_volume_days", "топ-10 діб за обсягом", "candles_1d", {}, {"_id": 0,
   "date": 1, "volume": 1, "close": 1}, [("volume", -1)], 10)
# Q3: вузький діапазон ts (одна година 12.03.2020) у хвилинній колекції (індекс
   ts_1), сортування за low зростаючи, limit 5
# -> 5 найнижчих мінімумів під час обвалу «чорного четверга»
run("q3_covid_hour", "12.03.2020 12:00-13:00 UTC, 5 найнижчих low", "candles_1m",
    {"ts": {"$gte": U(2020, 3, 12, 12), "$lt": U(2020, 3, 12, 13)}}, {"_id": 0,
   "ts": 1, "low": 1, "close": 1, "volume": 1}, [("low", 1)], 5)
# Q4: відсікаємо нульовий обсяг ($gt:0), сортуємо за volume спаданням, limit 5 ->
   хвилини з рекордним обсягом (індекс volume_-1)
run("q4_top5_volume_minutes", "5 хвилин з найбільшим обсягом", "candles_1m",
   {"volume": {"$gt": 0}},
    {"_id": 0, "ts": 1, "close": 1, "volume": 1}, [("volume", -1)], 5)
```

```
# Q5: запит до колекції з результатами Spark (ПЗ2): повні доби (is_complete=1) з
   падінням ret_1d<-10 %, від найгіршої; limit 10
run("q5_worst_days_spark", "повні доби з ret_1d<-0,10 (результати Spark)",
   "spark_daily_features",
    {"is_complete": 1, "ret_1d": {"$lt": -0.10}}, {"_id": 0, "date": 1, "close":
   1, "ret_1d": 1, "rv": 1}, [("ret_1d", 1)], 10)
# Q6: пошук неповних діб (менше 1440 хвилин) із проєкцією лише дати та кількості
   хвилин; без sort і limit
run("q6_incomplete_days", "неповні доби (n_minutes<1440)", "candles_1d",
   {"n_minutes": {"$lt": 1440}},
    {"_id": 0, "date": 1, "n_minutes": 1})
# Q7: count_documents для хвилин з нульовим обсягом (повертається одне число, а
   не документи)
run("q7_zero_volume_count", "кількість хвилин з нульовим обсягом", "candles_1m",
   {"volume": 0}, count_only=True)
# Q8: комбінований логічний фільтр: $and із рівністю (year), $in (month 11 або
   12) та flat_share=0; сортування за датою, limit 5
run("q8_in_or", "доби листопада-грудня 2022 з flat_share=0 ($in + $and)",
   "spark_daily_features",
    {"$and": [{"year": 2022}, {"month": {"$in": [11, 12]}}, {"flat_share": 0}]},
   {"_id": 0, "date": 1, "flat_share": 1, "volume": 1}, [("date", 1)], 5)
# зберігаємо зведену таблицю часу виконання всіх запитів
save_csv("pz4_queries_summary", summary)
```

A.5 Файл pz4_aggs.py – конвеєри агрегації

```
# -*- coding: utf-8 -*-
"""Завдання 5: конвеєри агрегації ($match/$sort/$group/$project/$bucket).
   Результати -> results/pz4_agg_*.csv"""
import sys, time, json
import pandas as pd
from bson import json_util
sys.path.insert(0, __import__("os").path.dirname(__file__))
from pz4_common import *

db = get_db()
ONLY = set(sys.argv[1:])   # напр. `python3 pz4_aggs.py A3` - виконати лише
   вибрані конвеєри
want = lambda k: not ONLY or k in ONLY


def agg(name, coll, pipe, **kw):
    t = time.perf_counter()
    res = list(db[coll].aggregate(pipe, allowDiskUse=True, **kw))
    s = time.perf_counter() - t
    append_timing(f"agg_{name}", s, f"{coll}, {len(res)} рядків")
    print(name, len(res), f"{s:.1f} c", flush=True)
    return res, s


if want("A1"):
    # A1. Місячні OHLC зі свічок 1 хв: $sort за ts + $first/$last
    a1 = [{"$sort": {"ts": 1}},
          {"$group": {"_id": {"y": {"$year": "$ts"}, "m": {"$month": "$ts"}},
  "open": {"$first": "$open"}, "high": {"$max": "$high"},
                      "low": {"$min": "$low"}, "close": {"$last": "$close"},
  "volume": {"$sum": "$volume"}, "n": {"$sum": 1}}},
          {"$sort": {"_id": 1}},
          {"$project": {"_id": 0, "year": "$_id.y", "month": "$_id.m", "open": 1,
  "high": 1, "low": 1, "close": 1, "volume": 1, "n": 1}}]
    res, t_idx = agg("A1_monthly_ohlc_with_ts_index", "candles_1m", a1)
    pd.DataFrame(res).to_csv(f"{RES}/pz4_agg_monthly_ohlc.csv", index=False)
    ex = db.command({"explain": {"aggregate": "candles_1m", "pipeline": a1,
  "cursor": {}, "allowDiskUse": True}, "verbosity": "executionStats"})
    json.dump(json.loads(json_util.dumps(ex)),
  open(f"{EXPL}/A1_monthly_ohlc_explain.json", "w"), indent=1)
    # той самий конвеєр без індексу на ts (примусовий $natural): справедливе
  порівняння часу
    res2, t_nat = agg("A1_monthly_ohlc_natural_hint", "candles_1m", a1,
  hint={"$natural": 1})
```

```
    save_csv("pz4_agg_A1_timing", [dict(variant="з індексом ts_1 ($sort з
  індексу)", seconds=round(t_idx, 2)),
                                   dict(variant="без індексу ($natural, зовнішнє
  сортування)", seconds=round(t_nat, 2))])

    # перевірка коректності A1 проти pandas-ресемплінгу з btc_1d.parquet
    d = pd.read_parquet(f"{W}/data/btc_1d.parquet").set_index("datetime_utc")
    m = d.resample("MS").agg({"Open": "first", "High": "max", "Low": "min",
  "Close": "last", "Volume": "sum"})
    m["year"], m["month"] = m.index.year, m.index.month
    a = pd.DataFrame(res).merge(m, on=["year", "month"])
    chk = dict(months=len(a), max_abs_open=(a.open - a.Open).abs().max(),
  max_abs_close=(a.close - a.Close).abs().max(),
               max_abs_high=(a.high - a.High).abs().max(), max_abs_low=(a.low -
  a.Low).abs().max(),
               max_rel_volume=((a.volume - a.Volume).abs() /
  a.Volume.clip(lower=1e-9)).max())
    save_csv("pz4_agg_A1_validation", [chk]); print(chk)

if want("A2"):
    # A2. Обсяг за годиною доби (UTC), 2018+ (ринок після 2017 вже ліквідний)
    a2 = [{"$match": {"ts": {"$gte": __import__("datetime").datetime(2018, 1, 1,
  tzinfo=__import__("datetime").timezone.utc)}}},
          {"$group": {"_id": {"$hour": "$ts"}, "total_volume": {"$sum":
  "$volume"}, "avg_volume_per_min": {"$avg": "$volume"}, "minutes": {"$sum":
  1}}},
          {"$sort": {"_id": 1}}, {"$project": {"_id": 0, "hour": "$_id",
  "total_volume": 1, "avg_volume_per_min": 1, "minutes": 1}}]
    res, _ = agg("A2_volume_by_hour", "candles_1m", a2)
    pd.DataFrame(res).to_csv(f"{RES}/pz4_agg_volume_by_hour.csv", index=False)

if want("A3"):
    # A3. Річна волатильність із результатів Spark (ПЗ2): лише повні доби
    a3 = [{"$match": {"is_complete": 1}},
          {"$group": {"_id": "$year", "days": {"$sum": 1}, "avg_rv": {"$avg":
  "$rv"}, "avg_hl_rel": {"$avg": "$hl_rel"},
                      "min_ret_1d": {"$min": "$ret_1d"}, "max_ret_1d": {"$max":
  "$ret_1d"}, "total_volume": {"$sum": "$volume"},
                      "avg_flat_share": {"$avg": "$flat_share"}}},
          {"$sort": {"_id": 1}}, {"$project": {"_id": 0, "year": "$_id", "days":
  1, "avg_rv": 1, "avg_hl_rel": 1, "min_ret_1d": 1,
                                              "max_ret_1d": 1, "total_volume":
  1, "avg_flat_share": 1}}]
    res, _ = agg("A3_yearly_volatility", "spark_daily_features", a3)
    pd.DataFrame(res).to_csv(f"{RES}/pz4_agg_yearly_volatility.csv", index=False)

if want("A4"):
    # A4. $bucket: розподіл діб за ціною закриття
    a4 = [{"$bucket": {"groupBy": "$close", "boundaries": [0, 10, 100, 1000,
  10000, 50000, 100000, 1000000], "default": "інше",
                       "output": {"days": {"$sum": 1}, "avg_volume": {"$avg":
  "$volume"}, "first_date": {"$min": "$date"}}}}]
    res, _ = agg("A4_bucket_close", "candles_1d", a4)
    rows = [dict(bucket_from=r["_id"], days=r["days"],
  avg_volume=r["avg_volume"], first_date=r["first_date"].strftime("%Y-%m-%d"))
  for r in res]
    save_csv("pz4_agg_bucket_close", rows)

if want("A5"):
    # A5. Річний підсумок з candles_1d (контроль A1): сума обсягу та діапазон цін
  за рік
    a5 = [{"$group": {"_id": "$year", "volume": {"$sum": "$volume"}, "max_high":
  {"$max": "$high"}, "min_low": {"$min": "$low"}}},
          {"$sort": {"_id": 1}}]
    res, _ = agg("A5_yearly_volume", "candles_1d", a5)
    pd.DataFrame(res).rename(columns={"_id":
  "year"}).to_csv(f"{RES}/pz4_agg_yearly_volume.csv", index=False)
```

A.6 Файл pz4_indexes.py – індекси та порівняння до/після

```
# -*- coding: utf-8 -*-
"""Завдання 6: індекси та порівняння виконання запитів до/після (explain
   executionStats).
Умови порівняння: той самий запит, один warm-up запуск, далі 5 вимірювань; у
   таблицю йде медіана.
Для 'до' усі вторинні індекси колекції видаляються, для 'після' створюється рівно
   один індекс."""
import sys, time, json, statistics, datetime as dt
from pz4_common import experiment
from bson import json_util
sys.path.insert(0, __import__("os").path.dirname(__file__))
from pz4_common import *
from pz4_common import drop_secondary, measure, experiment

db = get_db()
U = lambda *a: dt.datetime(*a, tzinfo=dt.timezone.utc)
REPS = 5


rows = []
# E1: діапазон за часом, одна доба (1440 документів)
rows += experiment("E1_range_day", "candles_1m", {"ts": {"$gte": U(2021, 5, 19),
   "$lt": U(2021, 5, 20)}}, [("ts", 1)], 0,
                   [("ts", 1)], "діапазон ts: 19.05.2021 (1440 хв)")
# E2: діапазон за часом, місяць (~44 640 документів)
rows += experiment("E2_range_month", "candles_1m", {"ts": {"$gte": U(2021, 5, 1),
   "$lt": U(2021, 6, 1)}}, [("ts", 1)], 0,
                   [("ts", 1)], "діапазон ts: травень 2021 (44 640 хв)")
# E3: топ-10 хвилин за обсягом (сортування + limit)
rows += experiment("E3_top_volume", "candles_1m", {}, [("volume", -1)], 10,
   [("volume", -1)], "топ-10 хвилин за volume")
# E4: складений індекс на добових даних: 2021 рік, обсяг вище 90-го перцентиля
   2021 року
vols = sorted(d["volume"] for d in db.candles_1d.find({"year": 2021}, {"volume":
   1}))
X = round(vols[int(0.9 * len(vols))], 2)
json.dump({"X_volume_p90_2021": X}, open(f"{RES}/pz4_index_params.json", "w"))
rows += experiment("E4_year_volume", "candles_1d", {"year": 2021, "volume":
   {"$gt": X}}, [("volume", -1)], 0,
                   [("year", 1), ("volume", -1)], f"year=2021 та volume>{X}, sort
   volume desc")
# E5: bucket pattern: усі 24 години доби 19.05.2021 – плоска колекція 1h проти
   документа-доби
drop_secondary("candles_1h"); drop_secondary("candles_1d_buckets")
rows += experiment("E5a_flat_hours", "candles_1h", {"ts": {"$gte": U(2021, 5,
   19), "$lt": U(2021, 5, 20)}}, [("ts", 1)], 0,
                   [("ts", 1)], "24 години доби: плоска candles_1h")
rows += experiment("E5b_bucket_day", "candles_1d_buckets", {"date": U(2021, 5,
   19)}, None, 0, [("date", 1)],
                   "24 години доби: один документ-доба (bucket)")
# лишаємо корисні індекси для подальших запитів/агрегацій
db.candles_1m.create_index([("ts", 1)]); db.candles_1m.create_index([("volume", -
   1)])
db.spark_daily_features.create_index([("date", 1)]);
   db.candles_1d.create_index([("date", 1)])
db.candles_1d.create_index([("year", 1), ("volume", -1)])
for r in rows:
    r["explain_ms_runs"] = json.dumps(r["explain_ms_runs"]); r["wall_ms_runs"] =
   json.dumps(r["wall_ms_runs"])
save_csv("pz4_index_comparison", rows)
idx = []
for c in db.list_collection_names():
    if c.endswith("_ts") or c.startswith("system"): continue
    st = db.command("collStats", c)
    for n, sz in st["indexSizes"].items(): idx.append(dict(collection=c, index=n,
   size_MiB=round(sz / 2**20, 3)))
save_csv("pz4_index_sizes", idx)
print("OK")
```

А.7 Файл pz4_anomalies.py – імпорт аномалій ПЗ3, індекс і $lookup з ознаками Spark

```
# -*- coding: utf-8 -*-
"""Завдання 7 (частина ПЗ3): колекція anomalies, індекси, запити та $lookup зі
   spark_daily_features.
Скрипт перезапускний: читає актуальний work/pz3/results/pz3_anomalies.csv,
   перезавантажує колекцію
та пише власні результати results/pz4_anom_*.csv, explain/anom_*.json.
   Обов'язкові колонки: timestamp, date, score; решта (v2: z_vol, sigma_ret,
   med_vol, rule_version тощо) завантажується як є."""
import sys, time, json, math, datetime as dt
import pandas as pd
from bson import json_util
sys.path.insert(0, __import__("os").path.dirname(__file__))
from pz4_common import *
from pz4_common import experiment, drop_secondary

db = get_db()
SRC = sys.argv[1] if len(sys.argv) > 1 else f"{W}/pz3/results/pz3_anomalies.csv"
   # шлях до CSV можна передати аргументом
U = lambda *a: dt.datetime(*a, tzinfo=dt.timezone.utc)

# ---------- 1. завантаження ----------
df = pd.read_csv(SRC)
cols = list(df.columns)
t0 = time.perf_counter()
db.drop_collection("anomalies")
docs = []
for r in df.to_dict("records"):
    d = {}
    for k, v in r.items():
        if isinstance(v, float) and math.isnan(v): v = None            # NaN ->
   null
        elif hasattr(v, "item"): v = v.item()
        d[k] = v
    if "ts" in r: d["ts_epoch"] = int(r["ts"])                              #
   Int (секунди Unix)
    d["ts"] =
   dt.datetime.fromisoformat(str(r["timestamp"])).replace(tzinfo=dt.timezone.utc)
   # Date (UTC)
    d["date"] = dt.datetime.fromisoformat(str(r["date"])
   [:10]).replace(tzinfo=dt.timezone.utc)
    d["year"] = d["date"].year
    del d["timestamp"]
    docs.append(d)
for i in range(0, len(docs), 20000): db.anomalies.insert_many(docs[i:i + 20000],
   ordered=False)
load_s = time.perf_counter() - t0
n = db.anomalies.count_documents({})
append_timing("load_anomalies", load_s, f"{n} docs з {SRC}")
save_csv("pz4_anom_load", [dict(source=SRC, csv_rows=len(df), mongo_docs=n,
   columns="|".join(cols), load_s=round(load_s, 2))])
print("loaded", n, cols)

# ---------- 2. типи полів і пропуски ----------
types = []
for f in [k for k in db.anomalies.find_one().keys() if k != "_id"]:
    for r in db.anomalies.aggregate([{"$group": {"_id": {"$type": "$" + f}, "n":
   {"$sum": 1}}}]):
        types.append(dict(field=f, bson_type=r["_id"], docs=r["n"]))
save_csv("pz4_anom_field_types", types)

# ---------- 3. індекс {date:1, score:-1}: до/після ----------
rows = experiment("ANOM1_year_top_score", "anomalies", {"date": {"$gte": U(2021,
   1, 1), "$lt": U(2022, 1, 1)}}, [("score", -1)], 20,
                  [("date", 1), ("score", -1)], "аномалії 2021, топ-20 за score")
rows += experiment("ANOM2_day", "anomalies", {"date": U(2020, 3, 12)}, [("score",
   -1)], 0, [("date", 1), ("score", -1)],
```

```
                    "аномалії доби 12.03.2020, за score")
for r in rows:
    r["explain_ms_runs"] = json.dumps(r["explain_ms_runs"]); r["wall_ms_runs"] =
   json.dumps(r["wall_ms_runs"])
save_csv("pz4_anom_index_comparison", rows)
db.anomalies.create_index([("date", 1), ("score", -1)]);
   db.anomalies.create_index([("year", 1)])
save_csv("pz4_anom_indexes", [dict(index=k, key=str(v["key"])) for k, v in
   db.anomalies.index_information().items()])

# ---------- 4. агрегації ----------
def agg(name, coll, pipe):
    t = time.perf_counter(); res = list(db[coll].aggregate(pipe,
   allowDiskUse=True)); s = time.perf_counter() - t
    append_timing(f"agg_{name}", s, f"{coll}, {len(res)} рядків"); print(name,
   len(res), f"{s:.2f} c"); return res

# anomalies per year (+ частка від кількості хвилин не рахується, лише кількість)
res = agg("ANOM_per_year", "anomalies", [{"$group": {"_id": "$year", "n":
   {"$sum": 1}, "avg_score": {"$avg": "$score"},
                                                    "max_score": {"$max":
   "$score"}, "days": {"$addToSet": "$date"}}},
                                          {"$project": {"_id": 0, "year": "$_id",
   "n": 1, "avg_score": 1, "max_score": 1, "days": {"$size": "$days"}}},
                                          {"$sort": {"year": 1}}])
pd.DataFrame(res).to_csv(f"{RES}/pz4_anom_per_year.csv", index=False)

# $lookup: доба з аномаліями -> ознаки Spark (П32)
look = [{"$group": {"_id": "$date", "n_anom": {"$sum": 1}, "max_score": {"$max":
   "$score"}}},
        {"$lookup": {"from": "spark_daily_features", "localField": "_id",
   "foreignField": "date", "as": "f"}},
        {"$unwind": "$f"},
        {"$project": {"_id": 0, "date": "$_id", "n_anom": 1, "max_score": 1,
   "rv": "$f.rv", "hl_rel": "$f.hl_rel",
                      "ret_1d": "$f.ret_1d", "volume": "$f.volume", "flat_share":
   "$f.flat_share"}},
        {"$sort": {"n_anom": -1}}]
res = agg("ANOM_lookup_features", "anomalies", look)
out = pd.DataFrame(res); out["date"] =
   pd.to_datetime(out["date"]).dt.strftime("%Y-%m-%d")
out.to_csv(f"{RES}/pz4_anom_lookup_full.csv", index=False)
out.head(15).to_csv(f"{RES}/pz4_anom_lookup_top15.csv", index=False)
# підсумок: доби з аномаліями проти решти за середнім rv та обсягом
allf = pd.DataFrame(list(db.spark_daily_features.find({"is_complete": 1}, {"_id":
   0, "date": 1, "rv": 1, "volume": 1, "hl_rel": 1})))
allf["date"] = pd.to_datetime(allf["date"]).dt.tz_localize(None).dt.strftime("%Y-
   %m-%d")
flag = allf.date.isin(set(out.date))
save_csv("pz4_anom_days_vs_rest", [dict(group=g, days=int(m.sum()),
   mean_rv=allf[m].rv.mean(), median_rv=allf[m].rv.median(),
                                        mean_hl_rel=allf[m].hl_rel.mean(),
   median_volume=allf[m].volume.median())
                                   for g, m in (("доби з аномаліями", flag),
   ("решта діб", ~flag))])
# розподіл score ($bucket) і за правилами
if "rule" in cols:
    res = agg("ANOM_by_rule", "anomalies", [{"$group": {"_id": "$rule", "n":
   {"$sum": 1}, "avg_score": {"$avg": "$score"}}}, {"$sort": {"n": -1}}])
    pd.DataFrame(res).rename(columns={"_id":
   "rule"}).to_csv(f"{RES}/pz4_anom_by_rule.csv", index=False)
res = agg("ANOM_top_dates", "anomalies", [{"$group": {"_id": "$date", "n":
   {"$sum": 1}}}, {"$sort": {"n": -1}}, {"$limit": 10}])
pd.DataFrame([dict(date=r["_id"].strftime("%Y-%m-%d"), n=r["n"]) for r in
   res]).to_csv(f"{RES}/pz4_anom_top_dates.csv", index=False)
print("OK")
```

A.8 Файл pz4_storage_units.py – розміри колекцій у байтах, МіБ і МБ

```
# -*- coding: utf-8 -*-
```

```
"""M1: співвідношення розмірів у БАЙТАХ; МіБ (2^20) і МБ (10^6) подано явно. ->
   results/pz4_storage_units.csv"""
import os, sys
sys.path.insert(0, os.path.dirname(__file__))
from pz4_common import *
db = get_db()
CSV_BYTES = os.path.getsize(f"{W}/data/raw/btcusd_1-min_data.csv")    # 396 849
   494 Б (вихідний CSV, data/README.md)
rows = [dict(object="CSV btcusd_1-min_data.csv", metric="файл", bytes=CSV_BYTES)]
for c in ["candles_1m", "candles_1m_ts", "candles_1h", "candles_1d",
   "candles_1d_buckets", "spark_daily_features", "anomalies"]:
    s = db.command("collStats", c)
    for m, k in (("size (BSON без стиснення; для TS - бакети)", "size"),
   ("storageSize (на диску)", "storageSize"), ("totalIndexSize",
   "totalIndexSize")):
        rows.append(dict(object=c, metric=m, bytes=s[k]))
for r in rows:
    r["MiB"] = round(r["bytes"] / 2**20, 2); r["MB"] = round(r["bytes"] / 1e6, 2)
    r["ratio_to_csv_bytes"] = round(r["bytes"] / CSV_BYTES, 3)
    r["loadavg_1m"] = f"{os.getloadavg()[0]:.2f}"
save_csv("pz4_storage_units", rows)
for r in rows: print(r)
print("candles_1m size / CSV =", db.command("collStats", "candles_1m")["size"] /
   CSV_BYTES)
```

A.9 Файл pz4_flat_vs_ts.py – запити до плоскої і time-series колекцій

```
# -*- coding: utf-8 -*-
"""M4: 3 запити на candles_1m (плоска, індекс ts_1) та candles_1m_ts (time-
   series): доба, місяць, місячний OHLC за 2021 рік.
Прогрів + медіана з 5 (wall-clock клієнта, повне читання результату). ->
   results/pz4_flat_vs_ts.csv"""
import os, sys, time, statistics, datetime as dt
sys.path.insert(0, os.path.dirname(__file__))
from pz4_common import *
db = get_db(); U = lambda *a: dt.datetime(*a, tzinfo=dt.timezone.utc)
def rng(a, b): return {"ts": {"$gte": a, "$lt": b}}
ohlc = lambda a, b: [{"$match": rng(a, b)}, {"$sort": {"ts": 1}},
    {"$group": {"_id": {"$month": "$ts"}, "open": {"$first": "$open"}, "high":
   {"$max": "$high"}, "low": {"$min": "$low"},
                "close": {"$last": "$close"}, "volume": {"$sum": "$volume"}}},
   {"$sort": {"_id": 1}}]
Q = [("доба 19.05.2021 (1440 хв)", lambda c: list(db[c].find(rng(U(2021, 5, 19),
   U(2021, 5, 20))))),
     ("місяць травень 2021 (44 640 хв)", lambda c: list(db[c].find(rng(U(2021, 5,
   1), U(2021, 6, 1))))),
     ("місячний OHLC за 2021 рік ($match+$sort+$group)", lambda c:
   list(db[c].aggregate(ohlc(U(2021, 1, 1), U(2022, 1, 1)), allowDiskUse=True)))]
rows = []
for name, fn in Q:
    out = {}
    for c in ("candles_1m", "candles_1m_ts"):
        fn(c)                                       # прогрів
        ts_, la = [], []
        for _ in range(5):
            la.append(os.getloadavg()[0]); t = time.perf_counter(); r = fn(c);
   ts_.append((time.perf_counter() - t) * 1000)
        out[c] = (statistics.median(ts_), ts_, len(r), la)
    rows.append(dict(query=name, flat_ms_median=round(out["candles_1m"][0], 1),
   ts_ms_median=round(out["candles_1m_ts"][0], 1),
                     ts_over_flat=round(out["candles_1m_ts"][0] /
   out["candles_1m"][0], 2), n_returned_flat=out["candles_1m"][2],
                     n_returned_ts=out["candles_1m_ts"][2],
   flat_runs_ms=str([round(x, 1) for x in out["candles_1m"][1]]),
                     ts_runs_ms=str([round(x, 1) for x in out["candles_1m_ts"]
   [1]]),
                     loadavg_1m=f"{min(out['candles_1m'][3] +
   out['candles_1m_ts'][3]):.2f}-{max(out['candles_1m'][3] + out['candles_1m_ts']
   [3]):.2f}"))
    print(rows[-1], flush=True)
```

```
rows[0]["indexes_flat"] = str(list(db.candles_1m.index_information())); rows[0]
  ["indexes_ts"] = str(list(db.candles_1m_ts.index_information()))
save_csv("pz4_flat_vs_ts", rows)
```

A.10 Файл pz4_write_cost.py – вартість індексів під час запису

```
# -*- coding: utf-8 -*-
"""М3: вартість індексів при записі. 200 000 документів (перші за часом),
  BATCH=20000, у scratch-колекцію:
(a) без вторинних індексів, (b) з ts_1 + volume_-1 створеними ДО вставки; (c) час
  побудови обох індексів ПІСЛЯ вставки.
Медіана з 3 запусків; scratch-колекція видаляється наприкінці. ->
  results/pz4_write_cost.csv"""
import os, sys, time, statistics
import pandas as pd
from bson.int64 import Int64
sys.path.insert(0, os.path.dirname(__file__))
from pz4_common import *
N, BATCH, RUNS = 200000, 20000, 3
db = get_db(); S = "scratch_write_cost"
df = pd.read_parquet(f"{W}/data/btc_1min.parquet").iloc[2_000_000:2_000_000 + N]
  # фрагмент ~2014 р. (ліквідний ринок)
ts = list(pd.DatetimeIndex(df["datetime_utc"]).tz_convert("UTC").to_pydatetime())
  if getattr(df["datetime_utc"].dt, "tz", None) \
   else
  list(pd.DatetimeIndex(df["datetime_utc"]).tz_localize("UTC").to_pydatetime())
un, o, h, l, c, v = (df[k].tolist() for k in ("Timestamp", "Open", "High", "Low",
  "Close", "Volume"))
mk = lambda a, b: [{"ts": ts[i], "unix": Int64(un[i]), "open": o[i], "high":
  h[i], "low": l[i], "close": c[i], "volume": v[i]} for i in range(a, b)]
def insert_all():
    t = 0.0
    for s in range(0, N, BATCH):
        docs = mk(s, min(s + BATCH, N)); t0 = time.perf_counter();
  db[S].insert_many(docs, ordered=False); t += time.perf_counter() - t0
    return t
res = {"no_index": [], "ts_1+volume_-1": [], "build_after": []}; load = []
for _ in range(RUNS):
    db.drop_collection(S); db[S].create_collection if False else None
    res["no_index"].append(insert_all()); load.append(os.getloadavg()[0])
    db.drop_collection(S); db[S].create_index([("ts", 1)]);
  db[S].create_index([("volume", -1)])
    res["ts_1+volume_-1"].append(insert_all()); load.append(os.getloadavg()[0])
    db.drop_collection(S); insert_all()
    t0 = time.perf_counter(); db[S].create_index([("ts", 1)]);
  db[S].create_index([("volume", -1)]);
  res["build_after"].append(time.perf_counter() - t0)
db.drop_collection(S)
assert S not in db.list_collection_names()
med = {k: statistics.median(x) for k, x in res.items()}
rows = []
for k, lab in (("no_index", "без вторинних індексів"), ("ts_1+volume_-1", "з
  індексами ts_1 і volume_-1 (створені до вставки)")):
    rows.append(dict(variant=lab, docs=N, batch_size=BATCH,
  insert_s_median=round(med[k], 3), insert_docs_per_s_median=round(N / med[k]),
                   runs_s=str([round(x, 3) for x in res[k]]),
  slowdown_pct=round((med[k] / med["no_index"] - 1) * 100, 1),
                   throughput_loss_pct=round((1 - med["no_index"] / med[k]) *
  100, 1)))
rows.append(dict(variant="побудова ts_1 + volume_-1 після вставки (200 000
  док.)", docs=N, batch_size=BATCH, insert_s_median=round(med["build_after"], 3),
                insert_docs_per_s_median="", runs_s=str([round(x, 3) for x in
  res["build_after"]]), slowdown_pct="", throughput_loss_pct=""))
for r in rows: r["loadavg_1m_during"] = f"{min(load):.2f}-{max(load):.2f}"
save_csv("pz4_write_cost", rows)
for r in rows: print(r)
```

A.11 Файл pz4_a1_repeat.py – повторні запуски A1 з індексом і без нього, explain

```
# -*- coding: utf-8 -*-
"""A1 (місячний OHLC по всіх 7,8 млн хвилин) з індексом ts_1 та без нього
   ($natural): 3 запуски кожного, медіана; explain обох;
usedDisk/spills зі стадії SORT; контроль: місячний OHLC з candles_1d (медіана з
   5)."""
import os, sys, time, json, statistics
from bson import json_util
sys.path.insert(0, os.path.dirname(__file__))
from pz4_common import *
db = get_db()
a1 = [{"$sort": {"ts": 1}},
      {"$group": {"_id": {"y": {"$year": "$ts"}, "m": {"$month": "$ts"}}, "open":
   {"$first": "$open"}, "high": {"$max": "$high"},
                  "low": {"$min": "$low"}, "close": {"$last": "$close"},
   "volume": {"$sum": "$volume"}, "n": {"$sum": 1}}},
      {"$sort": {"_id": 1}}]
assert "ts_1" in db.candles_1m.index_information()
def walk(o, key, out):
    if isinstance(o, dict):
        for k, v in o.items():
            if k == key: out.append(v)
            walk(v, key, out)
    elif isinstance(o, list):
        for v in o: walk(v, key, out)
    return out
rows = []
for label, hint in (("з індексом ts_1", None), ("без індексу ($natural)",
   {"$natural": 1})):
    kw = {"hint": hint} if hint else {}
    list(db.candles_1m.aggregate(a1[:1] + [{"$limit": 1000}], allowDiskUse=True,
   **kw))
    runs, la = [], []
    for _ in range(3):
        la.append(os.getloadavg()[0]); t = time.perf_counter(); r =
   list(db.candles_1m.aggregate(a1, allowDiskUse=True, **kw));
   runs.append(time.perf_counter() - t)
        print(label, round(runs[-1], 1), flush=True)
    ex = db.command({"explain": {"aggregate": "candles_1m", "pipeline": a1,
   "cursor": {}, "allowDiskUse": True, **kw}, "verbosity": "executionStats"})
    tag = "A1_monthly_ohlc_explain" if not hint else
   "A1_monthly_ohlc_natural_explain"
    json.dump(json.loads(json_util.dumps(ex)), open(f"{EXPL}/{tag}.json", "w"),
   indent=1)
    j = json_util.dumps(ex)
    plan = ex.get("stages") or []
    docs_ex = max(walk(json.loads(j), "totalDocsExamined", [0]))
    used = walk(json.loads(j), "usedDisk", []); spills = walk(json.loads(j),
   "spills", [])
    rows.append(dict(variant=label, groups=len(r),
   seconds_median=round(statistics.median(runs), 2), seconds_runs=str([round(x, 2)
   for x in runs]),
                     totalDocsExamined=docs_ex, usedDisk_any=any(used),
   usedDisk_values=str(used), spills_values=str(spills),
                     winning_plan=str(ex.get("queryPlanner",
   {}).get("winningPlan", {}).get("queryPlan", ex.get("stages", [{}])
   [0].get("$cursor", {}).get("queryPlanner", {}).get("winningPlan",
   ""))).replace("\n", " ")[:300],
                     loadavg_1m=f"{min(la):.2f}-{max(la):.2f}",
   explain_file=f"explain/{tag}.json"))
    print(rows[-1], flush=True)
# контроль: місячний OHLC з добових свічок
pipe = [{"$sort": {"date": 1}}, {"$group": {"_id": {"y": "$year", "m": "$month"},
   "open": {"$first": "$open"}, "high": {"$max": "$high"},
        "low": {"$min": "$low"}, "close": {"$last": "$close"}, "volume": {"$sum":
   "$volume"}}}, {"$sort": {"_id": 1}}]
list(db.candles_1d.aggregate(pipe)); ts_, la = [], []
```

```
for _ in range(5):
    la.append(os.getloadavg()[0]); t = time.perf_counter(); r =
   list(db.candles_1d.aggregate(pipe)); ts_.append((time.perf_counter() - t) *
   1000)
rows.append(dict(variant="місячний OHLC з candles_1d (5 429 док.), мс",
   groups=len(r), seconds_median=round(statistics.median(ts_) / 1000, 4),
                  seconds_runs=str([round(x / 1000, 4) for x in ts_]),
   totalDocsExamined=5429, usedDisk_any="", usedDisk_values="", spills_values="",
                  winning_plan="", loadavg_1m=f"{min(la):.2f}-{max(la):.2f}",
   explain_file=""))
save_csv("pz4_agg_A1_timing", rows)
```

A.12 Файл s18_explain_ts.js – план запиту E1 у mongosh

```
// Порівняння плану ДО (hint $natural = COLLSCAN) і ПІСЛЯ (за індексом) для
   одного й того ж запиту
const f = (e) => { const s = e.executionStats; let p =
   e.queryPlanner.winningPlan, st = [];
  while (p) { st.push(p.stage); p = p.inputStage }
  return { stage: st.reverse().join(' <- '), executionTimeMillis:
   s.executionTimeMillis,
           totalDocsExamined: s.totalDocsExamined, totalKeysExamined:
   s.totalKeysExamined, nReturned: s.nReturned } };
const q = () => db.candles_1m.find({ts:{$gte:ISODate('2021-05-19'),
   $lt:ISODate('2021-05-20')}}).sort({ts:1});
print('BEFORE (hint $natural):'); printjson(f(q().hint({$natural:
   1}).explain('executionStats')));
print('AFTER (index):'); printjson(f(q().explain('executionStats')));
```

A.13 Файл s19_explain_vol.js – план запиту E3 у mongosh

```
// Порівняння плану ДО (hint $natural = COLLSCAN) і ПІСЛЯ (за індексом) для
   одного й того ж запиту
const f = (e) => { const s = e.executionStats; let p =
   e.queryPlanner.winningPlan, st = [];
  while (p) { st.push(p.stage); p = p.inputStage }
  return { stage: st.reverse().join(' <- '), executionTimeMillis:
   s.executionTimeMillis,
           totalDocsExamined: s.totalDocsExamined, totalKeysExamined:
   s.totalKeysExamined, nReturned: s.nReturned } };
const q = () => db.candles_1m.find({}).sort({volume:-1}).limit(10);
print('BEFORE (hint $natural):'); printjson(f(q().hint({$natural:
   1}).explain('executionStats')));
print('AFTER (index):'); printjson(f(q().explain('executionStats')));
```

A.14 Файл s20_explain_anom.js – план запиту ANOM1 у mongosh

```
// Порівняння плану ДО (hint $natural = COLLSCAN) і ПІСЛЯ (за індексом) для
   одного й того ж запиту
const f = (e) => { const s = e.executionStats; let p =
   e.queryPlanner.winningPlan, st = [];
  while (p) { st.push(p.stage); p = p.inputStage }
  return { stage: st.reverse().join(' <- '), executionTimeMillis:
   s.executionTimeMillis,
           totalDocsExamined: s.totalDocsExamined, totalKeysExamined:
   s.totalKeysExamined, nReturned: s.nReturned } };
const q = () => db.anomalies.find({date:{$gte:ISODate('2021-01-01'),
   $lt:ISODate('2022-01-01')}}).sort({score:-1}).limit(20);
print('BEFORE (hint $natural):'); printjson(f(q().hint({$natural:
   1}).explain('executionStats')));
print('AFTER (index):'); printjson(f(q().explain('executionStats')));
```

A.15 Файл pz4_figures.py – побудова рисунків

```
# -*- coding: utf-8 -*-
"""Рисунки ПЗ4 (підписи українською, dpi 200) за CSV з results/."""
import pandas as pd, numpy as np, matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
import sys; sys.path.insert(0, __import__("os").path.dirname(__file__))
from pz4_common import RES, FIG
from matplotlib.ticker import FuncFormatter
# Розбірливість (правило 2a): рисунки вставляються шириною 16 см (6,3 дюйма),
   тому figsize ~6,7 дюйма, шрифти >= 8 pt;
# десятковий роздільник - кома (Ukrainian decimal comma), тисячі - вузький
   пробіл.
sys.path.insert(0, "/home/user/AVD/work/tools_report")
import plotstyle  # правило 2b
def dec(v, d=None):
    """Число -> рядок з десятковою комою (d знаків або %g)."""
    t = f"{v:.{d}f}" if d is not None else f"{v:g}"
    return t.replace(".", ",")
comma_fmt = FuncFormatter(lambda v, _: dec(v))
def commas(ax, which="y"):
    (ax.yaxis if which == "y" else ax.xaxis).set_major_formatter(comma_fmt)
C1, C2 = "#c0504d", "#2e75b6"
ic = pd.concat([pd.read_csv(f"{RES}/pz4_index_comparison.csv"),
   pd.read_csv(f"{RES}/pz4_anom_index_comparison.csv")])
names = {"E1_range_day": "ts, 1 доба\n(candles_1m)", "E2_range_month": "ts, 1
   місяць\n(candles_1m)", "E3_top_volume": "топ-10 volume\n(candles_1m)",
          "E4_year_volume": "year+volume\n(candles_1d)", "E5a_flat_hours": "24
   год: плоска\n(candles_1h)", "E5b_bucket_day": "24 год: bucket\n(1d_buckets)",
          "ANOM1_year_top_score": "аномалії 2021\ntop-20", "ANOM2_day": "аномалії
   доби\n12.03.2020"}


def pair(col, xlabel, fn, title, log=True):
    ex = list(names); y = np.arange(len(ex))[::-1]; h = .38
    b = [ic[(ic.experiment == e) & (ic.phase == "до")][col].iloc[0] for e in ex]
    a = [ic[(ic.experiment == e) & (ic.phase == "після")][col].iloc[0] for e in
   ex]
    floor = .5 if log else 0
    fig, ax = plt.subplots(figsize=(6.3, 4.7))
    r1 = ax.barh(y + h / 2, np.maximum(b, floor), h, label="без індексу
   (COLLSCAN)", color=C1)
    r2 = ax.barh(y - h / 2, np.maximum(a, floor), h, label="з індексом (IXSCAN)",
   color=C2)
    top = max(max(b), max(a))
    for r, v in list(zip(r1, b)) + list(zip(r2, a)):
        ax.text(max(v, floor) * (1.15 if log else 1) + (0 if log else top * .01),
   r.get_y() + r.get_height() / 2, dec(v), va="center", fontsize=9)
    if log: ax.set_xscale("log"); ax.set_xlim(right=top * 8)
    ax.xaxis.set_major_formatter(comma_fmt)
    ax.set_yticks(y); ax.set_yticklabels([names[e] for e in ex], fontsize=9);
   ax.set_xlabel(xlabel); ax.set_title(title)
    ax.legend(loc="lower right"); ax.grid(axis="y", visible=False)
    fig.tight_layout(); fig.savefig(f"{FIG}/{fn}", dpi=200); plt.close(fig)


# та сама метрика, що в таблиці 6: executionTimeMillis з explain (медіана з 5);
   повний час у Python - окремий стовпець CSV
pair("executionTimeMillis_explain_median", "executionTimeMillis, мс (медіана з 5,
   лог. шкала; 0 = менше 1 мс)", "fig_idx_time.png", "Час виконання запиту до та
   після індексу")
pair("totalDocsExamined", "переглянуто документів (лог. шкала)",
   "fig_idx_docs.png", "totalDocsExamined до та після індексації")

m = pd.read_csv(f"{RES}/pz4_agg_monthly_ohlc.csv"); m["d"] =
   pd.to_datetime(dict(year=m.year, month=m.month, day=1))
fig, ax = plt.subplots(figsize=(6.3, 3.4)); ax.bar(m.d, m.volume / 1e3, width=25,
   color=C2)
ax.set_ylabel("обсяг, тис. BTC"); ax.set_title("Місячний обсяг торгів (агрегація
   $group по candles_1m)"); commas(ax); fig.tight_layout();
   fig.savefig(f"{FIG}/fig_monthly_volume.png", dpi=200); plt.close(fig)

h = pd.read_csv(f"{RES}/pz4_agg_volume_by_hour.csv")
fig, ax = plt.subplots(figsize=(6.3, 3.4)); ax.bar(h.hour, h.avg_volume_per_min,
   color=C2)
ax.set_xticks(range(0, 24, 2)); commas(ax); ax.set_xlabel("година доби (UTC)");
   ax.set_ylabel("середній обсяг за хвилину, BTC")
```

```
ax.set_title("Обсяг за годиною доби, 2018 р. і пізніше ($hour у конвеєрі)");
   fig.tight_layout(); fig.savefig(f"{FIG}/fig_volume_by_hour.png", dpi=200);
   plt.close(fig)

a = pd.read_csv(f"{RES}/pz4_anom_per_year.csv")
fig, ax = plt.subplots(figsize=(6.3, 3.4)); ax.bar(a.year, a.n, color=C1)
for x, v in zip(a.year, a.n): ax.text(x, v, str(v), ha="center", va="bottom",
   fontsize=9)
ax.set_xticks(a.year[::2]); ax.set_ylabel("кількість аномальних хвилин");
   ax.set_title("Аномалії ПЗ3 у колекції anomalies за роками (агрегація MongoDB)")
fig.tight_layout(); fig.savefig(f"{FIG}/fig_anomalies_per_year.png", dpi=200);
   plt.close(fig)

s = pd.read_csv(f"{RES}/pz4_storage.csv").set_index("collection")
fig, ax = plt.subplots(1, 2, figsize=(6.3, 3.4))
cs = ["candles_1m", "candles_1m_ts"]; x = np.arange(2); w = .27
# для TS-колекції size вже відображає стиснені бакети (не "дані без стиснення");
   індекс TS - meta_1_ts_1, а не _id
for i, (c, l, col) in enumerate((("size_MiB", "size (BSON)", "#999"),
   ("storageSize_MiB", "storageSize (диск)", C2), ("totalIndexSize_MiB",
   "індекси", C1))):
    ax[0].bar(x + (i - 1) * w, s.loc[cs, c], w, label=l, color=col)
    for xx, v in zip(x + (i - 1) * w, s.loc[cs, c]): ax[0].text(xx, v, dec(v, 0
   if v >= 10 else 2), ha="center", va="bottom", fontsize=9)
ax[0].set_xticks(x); ax[0].set_xticklabels(["плоска", "time-series\n(індекс
   meta_1_ts_1)"]); ax[0].set_ylabel("МіБ"); ax[0].legend(fontsize=9);
   ax[0].set_title("Хвилинні свічки, 7,8 млн док.", fontsize=10);
   ax[0].set_ylim(top=ax[0].get_ylim()[1] * 1.15); commas(ax[0])
cs = ["candles_1h", "candles_1d_buckets"]
for i, (c, l, col) in enumerate((("size_MiB", "size (BSON)", "#999"),
   ("storageSize_MiB", "storageSize (диск)", C2))):
    ax[1].bar(np.arange(2) + (i - .5) * .35, s.loc[cs, c], .35, label=l,
   color=col)
    for xx, v in zip(np.arange(2) + (i - .5) * .35, s.loc[cs, c]): ax[1].text(xx,
   v, dec(v, 1), ha="center", va="bottom", fontsize=9)
ax[1].set_xticks(range(2)); ax[1].set_xticklabels(["плоска\n(130 276 док.)",
   "bucket «доба»\n(5 429 док.)"]); ax[1].set_ylabel("МіБ");
   ax[1].legend(fontsize=9); ax[1].set_title("Годинні свічки", fontsize=10);
   ax[1].set_ylim(top=ax[1].get_ylim()[1] * 1.3); commas(ax[1])
fig.tight_layout(); fig.savefig(f"{FIG}/fig_storage.png", dpi=200);
   plt.close(fig)

# M4: плоска проти time-series (медіана з 5, wall-clock клієнта)
import os
if os.path.exists(f"{RES}/pz4_flat_vs_ts.csv"):
    ft = pd.read_csv(f"{RES}/pz4_flat_vs_ts.csv")
    lab = ["1 доба\n(1 440 хв)", "1 місяць\n(44 640 хв)", "місячний OHLC\nза 2021
   рік"]
    fig, ax = plt.subplots(figsize=(6.3, 3.4)); x = np.arange(len(ft)); w = .38
    r1 = ax.bar(x - w / 2, ft.flat_ms_median, w, color=C2, label="плоска
   candles_1m (індекс ts_1)")
    r2 = ax.bar(x + w / 2, ft.ts_ms_median, w, color=C1, label="time-series
   candles_1m_ts")
    for r, v in list(zip(r1, ft.flat_ms_median)) + list(zip(r2,
   ft.ts_ms_median)): ax.text(r.get_x() + r.get_width() / 2, v * 1.1, dec(v, 1),
   ha="center", fontsize=9)
    ax.set_yscale("log"); ax.set_ylim(top=ft[["flat_ms_median",
   "ts_ms_median"]].values.max() * 6); ax.set_xticks(x); ax.set_xticklabels(lab)
    ax.set_ylabel("час у Python, мс (лог.)"); ax.set_title("Три запити: плоска і
   time-series колекції (медіана з 5)"); ax.legend(loc="upper left"); commas(ax)
    fig.tight_layout(); fig.savefig(f"{FIG}/fig_flat_vs_ts.png", dpi=200);
   plt.close(fig)

v = pd.read_csv(f"{RES}/pz4_agg_yearly_volatility.csv")
fig, ax = plt.subplots(figsize=(6.3, 3.4)); ax.bar(v.year, v.avg_rv, color=C2)
ax.set_xticks(v.year[::2]); commas(ax); ax.set_ylabel("середня rv");
   ax.set_title("Річна волатильність (агрегація по spark_daily_features, ПЗ2)")
fig.tight_layout(); fig.savefig(f"{FIG}/fig_yearly_volatility.png", dpi=200);
   plt.close(fig)
print("ok")
```

А.16 Файл pz4_screens_txt.py – отримання реального виводу команд для скриншотів

```
# -*- coding: utf-8 -*-
"""Генерує реальні тексти виводу mongosh/python для скріншотів
   (screenshots/txt/*.txt).
Кожен файл: перший рядок "$ команда", далі фактичний вивід команди. Потім
   make_screens.py рендерить PNG."""
import subprocess, os, sys
BASE = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
OUT = f"{BASE}/screenshots/txt"; os.makedirs(OUT, exist_ok=True)
MSH = "/home/user/AVD/work/tools/msh/node_modules/.bin/mongosh"
DB = "bigdata_db"


import shlex
ONLY = set(sys.argv[1:])   # якщо задано імена (s03 s04 ...), перегенеруються
   лише вони
os.environ["PATH"] = os.path.dirname(MSH) + ":" + os.environ["PATH"]


def trim(out, n=40):
    L = out.rstrip().split("\n")
    return out.rstrip() if len(L) <= n else "\n".join(L[:n - 1] + [f"… (ще
   {len(L) - n + 1} рядків)"])


def run(name, cmd, note=None):
    """Виконує РЕАЛЬНУ команду через bash (cwd = pz4/); заголовок "$" == ця сама
   команда, тіло == її вивід."""
    if ONLY and name.split("_")[0] not in ONLY: return
    r = subprocess.run(["bash", "-c", cmd], capture_output=True, text=True,
   timeout=900, cwd=BASE)
    out = trim(r.stdout + r.stderr)
    head = f"$ {cmd}" + (f"\n# {note}" if note else "")
    open(f"{OUT}/{name}.txt", "w", encoding="utf-8").write(f"{head}\n{out}\n")
    print("ok", name)


def sh(name, js, shown=None, db=DB):
    """mongosh --quiet --eval '<js>' db; заголовок == виконана команда (shown
   ігнорується)."""
    js = js.replace("'", '"')   # подвійні лапки в JS, щоб команда в оболонці
   була читабельною (без екранування '"'"')
    run(name, f"mongosh --quiet --eval {shlex.quote(js)} {db}")


def shf(name, jsfile, note=None, db=DB):
    """mongosh --quiet db --file code/<jsfile>.js"""
    run(name, f"mongosh --quiet {db} --file code/{jsfile}", note)




# --- 2. База і колекції ---
sh("s01_show_dbs", "show dbs", db="admin")
sh("s02_collections", "db.getCollectionNames().filter(n => !
   n.startsWith('system.'))")
sh("s03_counts", "db.getCollectionNames().filter(n=>!
   n.startsWith('system.')).forEach(n => print(n.padEnd(22),
   db[n].countDocuments({})))")
# --- 3. Структура, типи, пропуски ---
sh("s04_sample_1m", "const P=(d)=>print(JSON.stringify(d));
   P(db.candles_1m.findOne({}, {_id:0})); P(db.candles_1m_ts.findOne({},
   {_id:0})); P(db.candles_1d_buckets.aggregate([{$limit:1},{$project:
   {_id:0,date:1,n_hours:1,hours:{$slice:['$hours',1]}}}]).toArray()[0])")
sh("s05_sample_spark",
   "printjson(db.spark_daily_features.findOne({is_complete:1},
```

```
   {_id:0,date:1,close:1,volume:1,rv:1,hl_rel:1,ret_1d:1,year:1,dow:1}))")
sh("s06_types", "printjson(db.candles_1m.aggregate([{$limit:1},{$project:
   {_id:0,ts:{$type:'$ts'},unix:{$type:'$unix'},open:{$type:'$open'},volume:
   {$type:'$volume'}}}]).toArray()[0]);
   printjson(db.candles_1d.aggregate([{$limit:1},{$project:{_id:0,date:
   {$type:'$date'},year:{$type:'$year'},n_minutes:
   {$type:'$n_minutes'}}}]).toArray()[0])")
sh("s07_missing", "print('1m null/missing close:',
   db.candles_1m.countDocuments({$or:[{close:null},{close:{$exists:false}}]}));
   print('1m volume==0:', db.candles_1m.countDocuments({volume:0})); print('spark
   ret_1d null:', db.spark_daily_features.countDocuments({ret_1d:null}));
   print('spark vol_30d null:',
   db.spark_daily_features.countDocuments({vol_30d:null}))")
# --- 4. Запити ---
sh("s08_q1", "db.candles_1d.find({date:{$gte:ISODate('2021-01-01'),
   $lt:ISODate('2022-01-01')},close:{$gt:60000}},
   {_id:0,date:1,close:1,volume:1}).sort({date:1}).limit(8)",
    "db.candles_1d.find({date:{$gte:ISODate('2021-01-01'),$lt:ISODate('2022-01-
   01')},close:{$gt:60000}},
   {_id:0,date:1,close:1,volume:1}).sort({date:1}).limit(8)")
sh("s09_q2", "db.candles_1d.find({},
   {_id:0,date:1,volume:1,close:1}).sort({volume:-1}).limit(10)")
sh("s10_q3", "db.candles_1m.find({ts:{$gte:ISODate('2020-03-12T12:00:00Z'),
   $lt:ISODate('2020-03-12T13:00:00Z')}},
   {_id:0,ts:1,low:1,close:1,volume:1}).sort({low:1}).limit(5).forEach(d =>
   print(d.ts.toISOString(), String(d.low).padStart(8),
   String(d.close).padStart(8), d.volume.toFixed(2).padStart(7)))")
sh("s11_q5", "db.spark_daily_features.find({is_complete:1,ret_1d:{$lt:-0.10}},
   {_id:0,date:1,close:1,ret_1d:1,rv:1}).sort({ret_1d:1}).limit(8)")
# --- 5. Агрегація ---
sh("s12_agg_ohlc", "db.candles_1m.aggregate([{$sort:{ts:1}},{$group:{_id:{y:
   {$year:'$ts'},m:{$month:'$ts'}},open:{$first:'$open'},high:{$max:'$high'},low:
   {$min:'$low'},close:{$last:'$close'},volume:{$sum:'$volume'}}},{$sort:{_id:1}},
   {$match:{'_id.y':2021,'_id.m':{$lte:4}}}],{allowDiskUse:true}).forEach(d =>
   print(d._id.y + '-' + String(d._id.m).padStart(2,'0'), d.open, d.high, d.low,
   d.close, Math.round(d.volume)))")
sh("s13_agg_hour", "db.candles_1m.aggregate([{$match:{ts:{$gte:ISODate('2018-01-
   01')}}},{$group:{_id:{$hour:'$ts'},total:{$sum:'$volume'},avg_min:
   {$avg:'$volume'}}},{$sort:{_id:1}},{$limit:6}])")
sh("s14_agg_vol", "db.spark_daily_features.aggregate([{$match:{is_complete:1}},
   {$group:{_id:'$year',days:{$sum:1},avg_rv:{$avg:'$rv'},avg_hl:
   {$avg:'$hl_rel'}}},{$sort:{_id:1}},{$match:{_id:{$gte:2017}}}])")
sh("s15_agg_bucket", "db.candles_1d.aggregate([{$bucket:
   {groupBy:'$close',boundaries:
   [0,10,100,1000,10000,50000,100000,1000000],default:'other',output:{days:
   {$sum:1},avg_volume:{$avg:'$volume'}}}}])")
sh("s16_agg_lookup", "db.anomalies.aggregate([{$group:{_id:'$date',n_anom:
   {$sum:1},max_score:{$max:'$score'}}},{$lookup:
   {from:'spark_daily_features',localField:'_id',foreignField:'date',as:'f'}},
   {$unwind:'$f'},{$project:
   {_id:0,date:'$_id',n_anom:1,max_score:1,rv:'$f.rv',volume:'$f.volume'}},{$sort:
   {n_anom:-1}},{$limit:6}]).forEach(d => print(d.date.toISOString().slice(0,10),
   String(d.n_anom).padStart(4), d.max_score.toFixed(2).padStart(6),
   d.rv.toFixed(4), Math.round(d.volume)))")
# --- 6. Індекси ---
for _c in ["candles_1m", "candles_1h", "candles_1d", "candles_1d_buckets",
   "spark_daily_features", "anomalies"]:
    pass
sh("s17_getindexes", "for (const c of
   ['candles_1m','candles_1h','candles_1d','candles_1d_buckets','spark_daily_featu
   res','anomalies']) { print('// ' + c); console.log(db[c].getIndexes()) }")
EXPL = lambda coll, flt, extra: (
    "const f=(e)=>{const s=e.executionStats;let
   p=e.queryPlanner.winningPlan,st=[];while(p){st.push(p.stage);p=p.inputStage};
   return {stage:st.reverse().join(' <-
   '),executionTimeMillis:s.executionTimeMillis,totalDocsExamined:s.totalDocsExami
   ned,totalKeysExamined:s.totalKeysExamined,nReturned:s.nReturned}};"
    f"const q=()=>db.{coll}.find({flt}){extra};"
    "print('BEFORE (hint $natural):');
   printjson(f(q().hint({$natural:1}).explain('executionStats')));"
    "print('AFTER (index):'); printjson(f(q().explain('executionStats')))")
```

```
shf("s18_explain_ts", "s18_explain_ts.js")
shf("s19_explain_vol", "s19_explain_vol.js")
shf("s20_explain_anom", "s20_explain_anom.js")
sh("s21_ts_stats", "const s=db.candles_1m_ts.stats();
   printjson({ns:s.ns,timeseries:
   {bucketCount:s.timeseries.bucketCount,avgBucketSize:s.timeseries.avgBucketSize}
   ,size:s.size,storageSize:s.storageSize}); const f=db.candles_1m.stats();
   printjson({ns:f.ns,count:f.count,size:f.size,storageSize:f.storageSize,avgObjSi
   ze:f.avgObjSize})")
# --- shell-виводи ---
run("s22_load_log", "grep -E '^\\{|^DONE|7200000| 0s$' results/load.log | tail -c
   4000",
    "журнал завантаження results/load.log (відфільтровані ключові рядки)")
run("s23_index_table", "python3 -c \"import pandas as pd;
   pd.set_option('display.width',200);
   t=pd.read_csv('results/pz4_index_comparison.csv');
   print(t[['experiment','phase','executionTimeMillis_explain_median','totalDocsEx
   amined','totalKeysExamined','nReturned','winning_plan']].to_string(index=False)
   )\"")
```

A.17 Файл make_screens.py – рендер виводу терміналу в PNG

```
# -*- coding: utf-8 -*-
"""(ПЗ4) Рендер реального виводу програми (screenshots/txt/*.txt) у PNG спільним
   рендером
tools_report/term_png.py (1600 px, 90 колонок, <=35 рядків, перенос з «↪»).
   Формат txt: рядок 1 "$ команда",
необов'язковий рядок 2 "# примітка", далі вивід; кінцевий рядок "… (ще N рядків)"
   означає скорочення на етапі генерації."""
import glob, os, re, sys
sys.path.insert(0, "/home/user/AVD/work/tools_report")
from term_png import render_terminal_png
BASE = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
for p in sorted(glob.glob(f"{BASE}/screenshots/txt/*.txt")):
    lines = open(p, encoding="utf-8").read().rstrip("\n").split("\n")
    cmd = lines[0][2:] if lines[0].startswith("$ ") else lines[0]
    rest = lines[1:]
    note = None
    if rest and rest[0].startswith("# "):
        note, rest = rest[0][2:], rest[1:]
    extra = 0
    if rest:
        m = re.match(r"^… \(ще (\d+) рядків\)$", rest[-1])
        if m:
            extra, rest = int(m.group(1)), rest[:-1]
    out = f"{BASE}/screenshots/term_{os.path.basename(p)[:-4]}.png"
    render_terminal_png(cmd, "\n".join(rest), out, note=note, tail=0 if extra
  else 10, extra_skipped=extra)
    print(out)
```